VOID HACKS() 8.0 · TRACK: ABHEDYA — CYBER SECURITY & DIGITAL FORENSICS

Backend Architecture & API Contract

सेतु अनुबंध

The implementation contract between the FastAPI backend, the detection and graph engines, the evidence and document engines, and the React frontend: 45 routes, one response envelope, a closed error-code set, the EvidenceObject, and the MVP build order.

VERSION	1.0 — implementation contract
STATUS	Frozen for v1.0 — route changes need a version bump
OWNER	M1 — Technical Lead
COMPANION TO	Product Requirements Document v2.1
ASSOCIATED ORG	Indore Police Commissionerate
EVENT	Void Hacks() 8.0
DATE	02 October 2026

[PS] problem statement · [DEC] team decision · [ASSUME] assumption · [DATA-PRELIM] unverified dataset figure

Version 1.0 · Implementation contract

This document is the binding interface between five parties: the **FastAPI backend**, the **detection engine**, the **graph engine**, the **evidence & document engine**, and the **React frontend**. It is derived from Product Requirements Document v2.1 (dataset-grounded) and from the actual hackathon dataset `VoidHacks8_MuleAccount_2M_Transactions.csv`.

If this document and the PRD disagree on a contract (route, payload, field name, status code), **this document wins**. If they disagree on a behaviour (threshold, algorithm, scoring rule), **the PRD wins** and this document must be corrected.

0.1 How to read this document

Marker	Meaning
[PS]	Derives from the official problem statement
[DEC]	Team engineering decision, changeable only by the Technical Lead
[ASSUME]	Assumption not yet confirmed — listed in PRD §29
[DATA-PRELIM]	Dataset figure from preliminary profiling, **not yet re-verified** by `make profile-check` (PRD §7.0)
MUST / MUST NOT	Non-negotiable. A violation is a bug, not a preference
SHOULD	Strong default; deviation requires a comment in the code saying why

0.2 Dataset facts the API is designed around

Fact	Value	API consequence
Transaction rows	2,000,000 **[DATA-PRELIM]**	Nothing in the hot path may scan the fact table
Unique accounts	≈24,873 **[DATA-PRELIM]**	Per-account state is small enough to hold fully in RAM
`Transaction_ID`	**NOT unique** (≈1,997,748 distinct) **[DATA-PRELIM]**	`source_row_id` is the evidence key; `/api/transactions/{id}` returns a **list**
Unique (sender→receiver) pairs	≈2.0 M (≈99.8% of rows) **[DATA-PRELIM]**	Edge aggregation is near-identity; CSR ≈128 MB both directions
Avg degree	≈80 out **and** ≈80 in	Every account both sends and receives → every response carries **both sides**
Observed window	2026-09-15 → 2026-09-29 **[DATA-PRELIM]**	Default time filters clamp to this range
Terminal signals	≈0.13% of rows, probably co-occurring **[ASSUME]**	Signals are returned as evidence, never as a verdict

0.3 Status of this contract

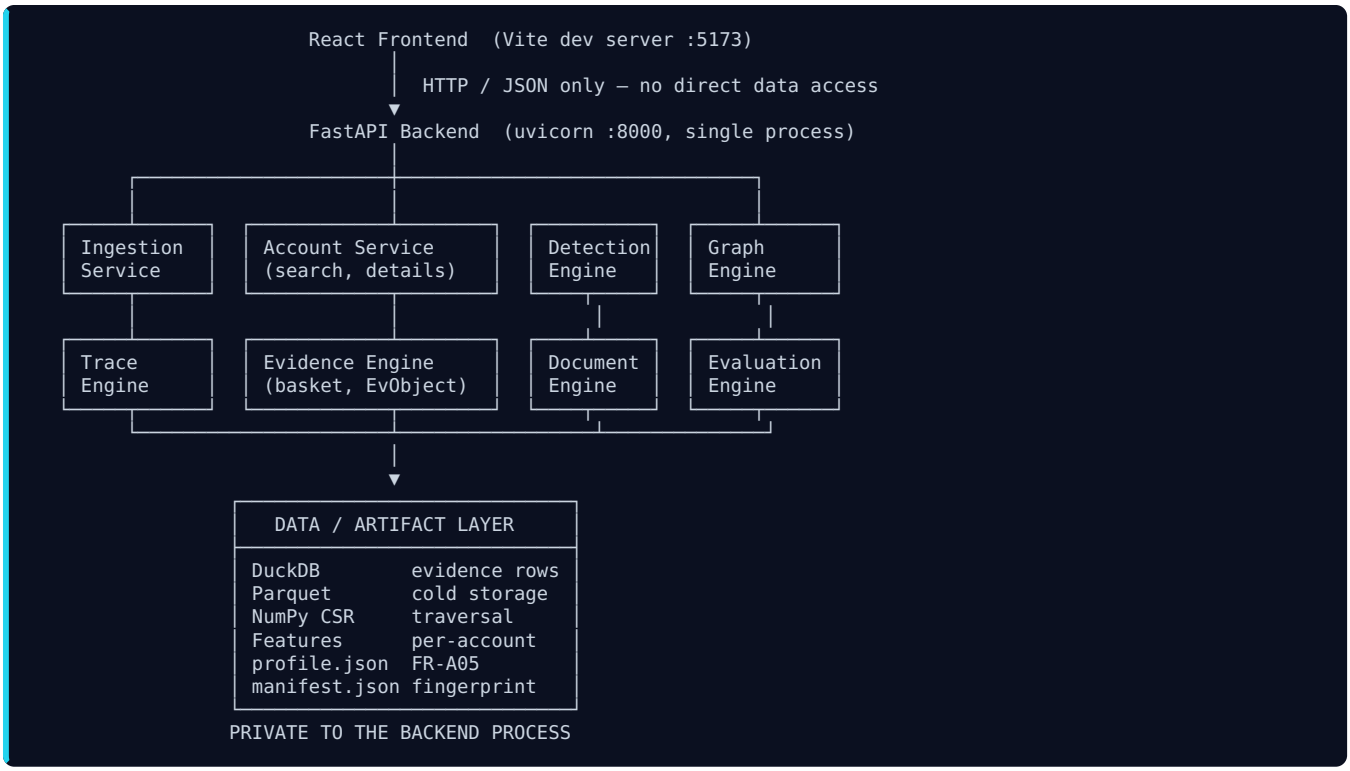
```
ROUTES              : FROZEN at v1.0 — adding a route needs a version bump
RESPONSE FIELDS     : additive changes allowed; removals/renames need a bump
NUMERIC VALUES      : every number in every example below is ILLUSTRATIVE
BENCHMARKS          : null until measured (section 22) — never a placeholder number
GROUND TRUTH        : unavailable (PRD Q1, OPEN) — section 21 returns a status, not a score
```

1. Backend Responsibilities

The backend owns **all** of the following. None of it may be re-implemented in the frontend.

#	Responsibility	Service	Phase
1	Dataset ingestion	Ingestion Service	P0
2	Dataset profiling (FR-A05)	Ingestion Service	P0
3	Data normalization	Ingestion Service	P0
4	Account indexing (`account_no` ↔ `account_idx`)	Ingestion Service	P0
5	Feature computation (dual-sided, ~45 features)	Detection Engine	P0
6	Mule detection (rule cards R1–R10)	Detection Engine	P0
7	Mule Risk Index (MRI) calculation	Detection Engine	P0
8	L1/L2/L3 classification (10 inputs, victim-relative)	Detection Engine	P0
9	Money-trail tracing (4-hop, value-attributed)	Trace Engine	P0
10	Graph / subgraph generation	Graph Engine	P0
11	Timeline filtering	Graph Engine	P1
12	Account search	Account Service	P0
13	Evidence collection (basket, EvidenceObject)	Evidence Engine	P1
14	Case diary generation	Document Engine	P1
15	Freeze requisition generation	Document Engine	P1
16	Evaluation / benchmark endpoints	Evaluation Engine	P2
17	Health / status monitoring	Core	P0

1.1 Service architecture



[DEC] The artifact layer has no network surface. DuckDB, Parquet, the CSR arrays, the feature frame and the manifest are process-local objects reachable only through `AppState`. There is no read-only SQL endpoint, no file-serving route into `data/`, and no "debug" passthrough — not even behind a flag.

1.2 Service responsibilities in one line each

Service	Owns	Must never
Ingestion	CSV → DuckDB + Parquet + CSR + features + `profile.json` + `manifest.json`; job lifecycle	Drop, merge or quarantine a source row
Account	Search, details, per-account statistics, transaction pagination	Compute risk or layer itself (reads the feature frame)
Detection	~45 dual-sided features, rule cards, MRI, bands, reasons, terminal evidence	Declare fraud; it produces scores and reasons
Graph	CSR construction, k-radius subgraph extraction, bounding, hub guard, rings/cycles	Return an unbounded subgraph
Trace	Victim-anchored, value-attributed, temporally-ordered traversal; layer assignment	Scan the fact table during a normal trace
Evidence	Basket persistence, `EvidenceObject` assembly, validators V1–V12	Emit a fact that has no `source_row_id`
Document	Case diary and freeze requisition rendering from a validated EvidenceObject	Let the LLM produce any factual token
Evaluation	Metrics, confusion matrix, per-layer breakdown, benchmark readout	Invent a number when labels or measurements are absent

1.3 Request lifecycle (every request, no exceptions)

```
1  Request ID middleware        → generate/propagate X-Request-ID
2  Timing middleware            → start perf_counter
3  Readiness guard              → 503 DATASET_NOT_LOADED if artifacts absent
4  Pydantic request validation  → 422 → mapped to our error envelope
5  Router → Service → AppState  (in-RAM reads; DuckDB only for evidence rows)
6  Pydantic response model      → serialise
7  Timing middleware            → meta.processing_ms
8  Envelope wrapper             → {success, data, meta}
```

[DEC] Steps 1, 2, 7 and 8 are middleware, not per-route code. No router may construct an envelope by hand.

---

2. Base Configuration

2.1 Development endpoints

Item	Value
Backend host	`127.0.0.1`
Backend port	`8000`
API prefix	`/api`
Frontend dev server	`http://localhost:5173`
Backend base URL	`http://localhost:8000`
OpenAPI (Swagger UI)	`http://localhost:8000/docs`
OpenAPI (ReDoc)	`http://localhost:8000/redoc`
OpenAPI JSON	`http://localhost:8000/openapi.json`
Health	`http://localhost:8000/api/health`

Demo mode binds `127.0.0.1` only. [DEC] The platform handles financial-crime data; it is local-first by design (PRD §23). Binding `0.0.0.0` requires an explicit `ABHEDYA_BIND_ALL=1` and is not used during judging.

2.2 Process model

```
uvicorn abhedya.main:app --host 127.0.0.1 --port 8000 --workers 1
```

[DEC] Exactly one worker. The CSR graph and the feature frame live in RAM; a second worker duplicates ≈1 GB and buys nothing, because traversal is already sub-100 ms. Concurrency comes from:

- `async def` routes for I/O-ish work,
- a bounded `ThreadPoolExecutor(max_workers=4)` for CPU-bound traversal and scoring (NumPy releases the GIL),
- a single background thread for the ingestion job.

2.3 CORS

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
    allow_credentials=False,
    allow_methods=["GET", "POST", "DELETE", "OPTIONS"],
    allow_headers=["Content-Type", "X-Request-ID"],
    expose_headers=["X-Request-ID", "X-Processing-Ms"],
)
```

`allow_origins=["*"]` is **forbidden** [DEC]. The list comes from `config.CORS_ORIGINS`.

2.4 Content types

Endpoint class	Response
Everything in this document	`application/json`

Endpoint class	Response
`GET /api/documents/{document_id}`	`application/pdf`, `application/vnd.openxmlformats-officedocument.wordprocessingml.document`, `application/json`, or `text/markdown`
`GET /api/documents/{document_id}/meta`	`application/json` (metadata about a file, so the frontend never has to parse a download)

Exactly one endpoint family returns bytes. Every other route returns the envelope of section 3.

2.5 Configuration surface (`abhedya/config.py`)

All tunables live in one module and are environment-overridable. Routers read `config`; they never hard-code a number. **[DEC]**

```
# ── paths ──────────────────────────────────────────────────────────
DATA_DIR            = Path(env("ABHEDYA_DATA_DIR", "data"))
RAW_DIR             = DATA_DIR / "raw"
ARTIFACT_DIR        = DATA_DIR / "artifacts"
DOCUMENT_DIR        = DATA_DIR / "documents"     # generated PDFs/DOCX only
UPLOAD_MAX_BYTES    = 400 * 1024 * 1024          # ~274 MB file + headroom

# ── server ─────────────────────────────────────────────────────────
API_PREFIX          = "/api"
CORS_ORIGINS        = ["http://localhost:5173", "http://127.0.0.1:5173"]
THREAD_POOL_SIZE    = 4
REQUEST_TIMEOUT_SEC = 30

# ── pagination & bounds ────────────────────────────────────────────
PAGE_SIZE_DEFAULT   = 50
PAGE_SIZE_MAX       = 500
SEARCH_LIMIT_MAX    = 100
GRAPH_MAX_NODES     = 800
GRAPH_MAX_EDGES     = 1_500
GRAPH_MAX_RADIUS    = 3
TRACE_MAX_NODES     = 800
TIMELINE_MAX_EVENTS = 5_000
BASKET_MAX_ITEMS    = 500

# ── analysis (mirrors PRD Appendix B — single source of truth) ─────
TRACE_MAX_HOPS        = 4
TRACE_WINDOW_SEC      = 21_600        # 6 h  (PRD §7.5; was 72 h in v2.0)
TRACE_WINDOW_PRESETS  = [3_600, 21_600, 86_400, 259_200]
TRACE_MIN_VALUE_FLOOR = 100.0         # max(floor, pct * initial_amount)
TRACE_MIN_VALUE_PCT   = 0.002
TRACE_TOPK_PER_NODE   = 15
HUB_DEGREE_CAP        = None          # None → max(500, p99_out_degree * 3)
MRI_BANDS             = {"LOW": 30, "MEDIUM": 60, "HIGH": 80}   # upper edges
PRESERVE_ALL_ROWS     = True
TXN_ID_IS_UNIQUE      = False
```

> **Rule 15 of section 28 restated:** if a reviewer greps the codebase for a magic number and finds it outside `config.py`, that is a defect ticket.

---

3. Standard Response Format

3.1 Success envelope

```
{
  "success": true,
  "data": { },
  "meta": {
    "request_id": "req_01JA8Z7K2M9QX4",
    "processing_ms": 123,
    "dataset_fingerprint": "sha256:7f3c…b41a",
    "api_version": "1.0"
  }
}
```

Field	Type	Always present	Notes
`success`	`bool`	yes	`true` only on 2xx
`data`	`object`	yes	Endpoint-specific payload; never a bare array — always a named object
`meta.request_id`	`string`	yes	Echoed in the `X-Request-ID` response header
`meta.processing_ms`	`int`	yes	Server-side wall time, measured by middleware
`meta.dataset_fingerprint`	`string`	yes once loaded	`sha256` prefix from `manifest.json`; lets the UI detect a dataset swap mid-session
`meta.api_version`	`string`	yes	`"1.0"`
`meta.warnings`	`string[]`	optional	Non-fatal notices, e.g. `"result truncated at max_nodes"`

[DEC] `data` is never a top-level array. `{"results": [...]}`, `{"transactions": [...]}`, `{"nodes": [...], "edges": [...]}`. This keeps every response additively extensible.

3.2 Error envelope

```
{
  "success": false,
  "error": {
    "code": "ACCOUNT_NOT_FOUND",
    "message": "Account was not found in the loaded dataset.",
    "details": { "account_no": "402938475610" },
    "hint": "Use /api/accounts/search for near matches."
  },
  "meta": {
    "request_id": "req_01JA8Z7K2M9QX4",
    "processing_ms": 4,
    "api_version": "1.0"
  }
}
```

Field	Type	Required	Notes
`error.code`	`string`	yes	From the closed set in section 23 — the frontend branches on this, never on `message`
`error.message`	`string`	yes	One human sentence, safe to show a user, **no stack trace, no path, no SQL**
`error.details`	`object`	yes (may be `{}`)	Machine-readable context: offending field, limits, counts

Field	Type	Required	Notes
`error.hint`	`string`	optional	What the caller can do next

3.3 HTTP status mapping

Status	Used for	Example codes
200	Success	—
201	Resource created	case created, basket created
202	Accepted, work continues asynchronously	ingestion started
400	Malformed/invalid request	`INVALID_ACCOUNT`, `INVALID_TIME_RANGE`
404	Named resource absent	`ACCOUNT_NOT_FOUND`, `CASE_NOT_FOUND`, `TRANSACTION_NOT_FOUND`
409	Conflicting state	`INGESTION_IN_PROGRESS`
413	Payload too large	upload exceeds `UPLOAD_MAX_BYTES`
422	Request bound-violation after validation	`TRACE_LIMIT_EXCEEDED`, `GRAPH_LIMIT_EXCEEDED`
500	Unexpected server fault	`INTERNAL_ERROR`
503	Not ready to serve	`DATASET_NOT_LOADED`

FastAPI's native 422 body is replaced. A `RequestValidationError` handler converts Pydantic errors into `INVALID_ACCOUNT` / `INVALID_TIME_RANGE` / `VALIDATION_ERROR` with the same envelope, so the frontend has exactly **one** error shape to parse.

3.4 Request ID

```
@app.middleware("http")
async def request_context(request: Request, call_next):
    rid = request.headers.get("X-Request-ID") or f"req_{ulid()}"
    request.state.request_id = rid
    t0 = time.perf_counter()
    try:
        response = await call_next(request)
    except AbhedyaError as e:                # typed, expected failures
        response = json_error(e, rid)
    except Exception:
        log.exception("unhandled", extra={"request_id": rid})
        response = json_error(InternalError(), rid)       # never leaks the trace
    ms = int((time.perf_counter() - t0) * 1000)
    response.headers["X-Request-ID"] = rid
    response.headers["X-Processing-Ms"] = str(ms)
    return response
```

Rules:

1. If the client sends `X-Request-ID`, it is reused; otherwise the server generates a ULID.
2. **The `request_id` appears in every log line for that request.** A judge asking "why did this account get flagged?" is answered by `grep` on one id.
3. **A raw Python exception MUST NOT reach the client.** The catch-all logs the traceback server-side and returns `INTERNAL_ERROR` with a generic message.

3.5 Pagination convention (all paginated endpoints)

```
"pagination": { "page": 1, "page_size": 50, "total": 143, "total_pages": 3, "has_next": true }
```

`page` is 1-based. `page_size` is clamped to `PAGE_SIZE_MAX`; a clamp adds `meta.warnings`. `total` is an exact count (cheap — DuckDB counts over an indexed predicate).

3.6 Common query parameter semantics

Parameter	Type	Rules
`account_no`	`string`	12 digits, `^[0-9]{12}$`, as a **string** — never a JSON number [DEC]
`start_time` / `end_time`	ISO-8601	Naive local time; `start_time < end_time`; clamped to the dataset window; violation → `INVALID_TIME_RANGE`
`amount`	`number`	Rupees, 2 dp, never a formatted string
`limit` / `page_size`	`int`	Clamped, never rejected silently
`direction`	enum	`in` · `out` · `all` (default `all`) — both sides always available (PRD §2.4)

> **Why account numbers are strings:** a 12-digit account number is not an integer quantity. JSON numbers lose leading zeros and, past 2^53, precision. Any client that parses an account number as a number is wrong. **[DEC]**

---

4. Dataset Ingestion APIs

4.1 `POST /api/ingest`

Load the transaction dataset and build every artifact the rest of the API depends on.

Request — `multipart/form-data`

Field	Type	Required	Notes
`file`	file	one of `file` / `path`	The CSV. Max `UPLOAD_MAX_BYTES`
`path`	string	one of `file` / `path`	Server-side path **relative to** `RAW_DIR` — used in the demo to avoid re-uploading 274 MB. Absolute paths and `..` are rejected
`force_rebuild`	bool	no (default `false`)	Rebuild artifacts even if the manifest fingerprint matches
`profile_only`	bool	no (default `false`)	Run stage `profiling` and stop — the FR-A05 "look before you leap" path

Response — 202 Accepted

```
{
  "success": true,
  "data": {
    "job_id": "ingest_20261001_001",
    "status": "started",
    "filename": "VoidHacks8_MuleAccount_2M_Transactions.csv",
    "size_bytes": 274019328,
    "sha256": "7f3c…b41a",
    "poll_url": "/api/ingest/status/ingest_20261001_001",
    "force_rebuild": false
  }
}
```

Rules

1. **MUST return within ~200 ms.** The frontend never blocks on a multi-second ingest; it polls `status`. [DEC]
2. Only **one** ingestion job may be active. A second attempt → `409 INGESTION_IN_PROGRESS` with `details.active_job_id`.
3. If `force_rebuild=false` and `manifest.json` records the same `sha256`, the job short-circuits to `completed` with `skipped_rebuild: true` — the demo reload costs nothing.
4. The upload is streamed to disk in chunks; it is **never** read into memory whole.
5. Validation before work begins: extension in `{.csv, .csv.gz}`, size under the cap, header parsed and matched against the expected 11 columns via the alias map (PRD §12). A header mismatch fails the job with `stage: "reading"` and the offending column list — it does not crash the server.

4.2 `GET /api/ingest/status/{job_id}`


```
{
  "success": true,
  "data": {
    "job_id": "ingest_20261001_001",
    "status": "running",
    "stage": "building_graph",
    "stage_index": 6,
    "stage_count": 9,
    "progress": 72,
    "rows_processed": 1440000,
    "total_rows": 2000000,
    "elapsed_ms": 38200,
    "stage_timings_ms": {
      "reading": 11200, "normalizing": 6400, "profiling": 4100,
      "indexing_accounts": 2600, "building_features": 9100
    },
    "warnings": [
      "2252 duplicate Transaction_ID values detected; all rows preserved (see /api/ingest/profile)"
    ],
    "error": null
  }
}
```


`status` values

Value	Meaning	Terminal
`queued`	Accepted, worker not started	no
`running`	In progress	no
`completed`	All artifacts built and loaded into `AppState`	yes
`failed`	Aborted; `error` holds a code + message; previous artifacts remain loaded	yes

`stage` values, in order

#	Stage	Produces
1	`reading`	Raw scan, header/alias resolution, row count
2	`normalizing`	Typed columns, `source_row_id`, `row_hash`, trimmed IDs, parsed timestamps
3	`profiling`	`profile.json` (FR-A05) **and the** `profile-check` **verdict**
4	`indexing_accounts`	`account_no` ↔ `account_idx` dictionaries
5	`building_features`	~45 dual-sided per-account features
6	`building_graph`	CSR forward + reverse, degree arrays, hub list
7	`scoring`	MRI, bands, reasons, terminal evidence, global (non-case) layer hints
8	`building_manifest`	`manifest.json` — fingerprint, row counts, config snapshot, timings
9	`completed`	`AppState` swapped atomically; `/api/health` flips to `healthy`

Rules

1. `progress` is an integer 0–100 derived from `(stage_index + intra_stage_fraction) / stage_count`. It is **monotonic**; it never goes backwards.
2. **Artifacts are published atomically at stage 9.** While a rebuild runs, queries continue to serve the previously loaded dataset; there is no window where the API answers from a half-built graph. [DEC]
3. `stage_timings_ms` is the raw material for `/api/benchmarks` — measured, never estimated.
4. On `failed`, `error` uses the section 23 code set and `details.stage` names where it died.

5. Unknown `job_id` → `404` with code `JOB_NOT_FOUND`. Job records are kept in memory for the process lifetime (a hackathon-scoped simplification, **[DEC]**, noted in §29 limitations).

4.3 `GET /api/ingest/profile`

Returns `profile.json` — the FR-A05 output. This is the **only** endpoint allowed to describe the dataset as a whole.


```
{
  "success": true,
  "data": {
    "generated_at": "2026-10-01T18:04:11",
    "source_file": "VoidHacks8_MuleAccount_2M_Transactions.csv",
    "dataset_fingerprint": "sha256:7f3c…b41a",

    "row_count": 2000000,
    "column_count": 11,
    "columns": ["Transaction_ID", "Timestamp", "Sender_Account", "Sender_IFSC",
                "Receiver_Account", "Receiver_IFSC", "Amount", "Payment_Mode",
                "Narration", "IP_Address", "Device_Type"],
    "unique_accounts": 24873,

    "unique_transaction_ids": 1997748,
    "duplicate_transaction_ids": {
      "row_surplus": 2252,
      "distinct_ids_repeated": 2252,
      "rows_affected": 4504,
      "repeat_histogram": [ { "repeat_count": 2, "n_ids": 2252, "n_rows": 4504 } ]
    },
    "exact_duplicate_rows": 0,
    "near_duplicate_rows": 4504,

    "missing_values": { "Narration": 0, "IP_Address": 0, "Device_Type": 0 },
    "malformed_ifsc": 0,
    "non_12_digit_accounts": 0,

    "date_range": { "start": "2026-09-15T00:00:11", "end": "2026-09-29T23:59:52", "span_days": 15 },

    "payment_modes": { "UPI": 1299844, "IMPS": 441199, "NEFT": 199134, "RTGS": 59823 },
    "device_types": { "Windows_Browser": 666004, "iOS": 665917, "Android": 665425,
                      "Web_Emulator": 1327, "Linux_Script": 1327 },

    "amount_statistics": {
      "min": 50.0, "p25": 312.4, "median": 899.35, "mean": 1760.88,
      "p75": 1904.0, "p99": 28410.5, "max": 499823.99,
      "by_payment_mode": { "RTGS": { "n": 59823, "median": 902.1, "min": 51.0, "max": 499823.99 } }
    },

    "suspicious_signals": {
      "foreign_ip_rows": 2654,
      "automation_device_rows": 2654,
      "crypto_narration_rows": 1327,
      "wallet_narration_rows": 1327,
      "any_signal_rows": 2654,
      "all_signal_rows": 2654,
      "cooccurrence_rate": 1.0,
      "accounts_touching_a_signal": 5308,
      "cooccurrence_matrix": {
        "foreign_ip_and_automation": 2654,
        "foreign_ip_and_narration": 2654,
        "automation_and_narration": 2654
      }
    },

    "graph_shape": {
      "unique_pairs": 1996771,
      "self_loops": 0,
      "out_degree": { "p50": 78, "p90": 104, "p99": 131, "max": 402, "mean": 80.41 },
      "in_degree":  { "p50": 78, "p90": 104, "p99": 130, "max": 388, "mean": 80.41 },
      "suggested_hub_degree_cap": 500
    },

    "verification": {
      "profile_check_run": true,
      "profile_check_passed": true,
      "mismatches": [],
      "expectation_source": "PRD v2.1 section 7.2"
    }
  }
}
```

Rules

1. **Every number here is measured from the CSV at ingest time.** No value in this payload may be copied from the PRD. The PRD holds expectations; this endpoint holds observations. [DEC]
2. `verification.profile_check_passed` is what the admin screen renders as VERIFIED / MISMATCH. If it is `false`, `mismatches` lists `{key, expected, actual, tolerance}` and the UI **must** show the banner — a silent mismatch is a P0 defect.
3. `duplicate_transaction_ids` reports **row surplus and affected rows separately**. They are different numbers and conflating them is the exact mistake v2.0 made.
4. `cooccurrence_rate` feeds the MRI C5 discount (PRD §15.1). A rate of `1.0` means the four terminal signals are one fact, not four.
5. Values are serialised as JSON numbers; counts are integers; no thousands separators, no `"₹"` prefixes — **formatting is the frontend's job**.
6. Before ingestion has ever run → `503 DATASET_NOT_LOADED`.

4.4 `GET /api/ingest/manifest`

```
{
  "success": true,
  "data": {
    "dataset_fingerprint": "sha256:7f3c…b41a",
    "source_file": "VoidHacks8_MuleAccount_2M_Transactions.csv",
    "ingested_at": "2026-10-01T18:04:52",
    "rows_in": 2000000,
    "rows_out": 2000000,
    "row_conservation_ok": true,
    "accounts": 24873,
    "edges": 1996771,
    "artifact_versions": { "schema": "2.1", "features": "2.1", "csr": "2.1" },
    "config_snapshot": { "TRACE_WINDOW_SEC": 21600, "TRACE_TOPK_PER_NODE": 15,
                         "MRI_BANDS": { "LOW": 30, "MEDIUM": 60, "HIGH": 80 } },
    "stage_timings_ms": { "reading": 11200, "normalizing": 6400 }
  }
}
```

`row_conservation_ok` implements **NFR-11**: `rows_out == rows_in == 2,000,000`. If it is ever `false`, the ingest job **MUST** fail rather than publish the artifacts. Losing a row silently is the failure mode this whole architecture exists to prevent.

---

5. Health API

5.1 `GET /api/health`

```
{
  "success": true,
  "data": {
    "status": "healthy",
    "api_version": "1.0",
    "uptime_sec": 1843,

    "dataset_loaded": true,
    "artifacts_loaded": true,
    "graph_loaded": true,
    "features_loaded": true,

    "accounts": 24873,
    "transactions": 2000000,
    "edges": 1996771,
    "dataset_fingerprint": "sha256:7f3c…b41a",
    "dataset_window": { "start": "2026-09-15T00:00:11", "end": "2026-09-29T23:59:52" },

    "profile_verified": true,
    "ground_truth_available": false,
    "llm_available": false,

    "ingest_job": { "active": false, "last_job_id": "ingest_20261001_001", "last_status": "completed" },
    "memory_mb": { "rss": 1042, "csr": 128, "features": 9 }
  }
}
```

`status`	Meaning	HTTP
`healthy`	Artifacts loaded, all queries available	200
`degraded`	Serving, but something optional is missing (e.g. `profile_verified: false`, LLM down)	200
`loading`	Ingestion running, previous artifacts may still be serving	200
`unavailable`	No dataset has ever been loaded	200

Rules

1. `/api/health` **MUST** answer 200 even when nothing is loaded — it is the frontend's bootstrap probe and a 503 here would be indistinguishable from "backend is down". The state lives in `data.status`. [DEC]
2. It must not touch DuckDB or the CSR arrays; it reads counters from `AppState`. Target: **< 5 ms**.
3. `llm_available` is a cached probe of the local Ollama socket, refreshed at most every 30 s. **`false` is a perfectly good answer** — the document path is template-first and fully functional without it (PRD §20).
4. `ground_truth_available` is the single switch that section 21 keys off.
5. The frontend polls this every 5 s while an ingest job is active, and every 60 s otherwise.

6. Account Search API

6.1 `GET /api/accounts/search`

Parameter	Type	Default	Rules
`q`	string	required	2–12 characters, digits only (prefix/substring of an account number)
`limit`	int	20	Clamped to `SEARCH_LIMIT_MAX` (100)
`min_risk`	float	—	Optional 0–100 filter
`layer`	int	—	Optional `1` | `2` | `3` filter (global hint layer, see 6.3)
`sort`	enum	`relevance`	`relevance` | `risk_desc` | `volume_desc`

Response


```
{
  "success": true,
  "data": {
    "query": "4029",
    "results": [
      {
        "account_no": "402938475610",
        "account_idx": 1234,
        "primary_ifsc": "HDFC0001234",
        "bank_code": "HDFC",
        "risk_score": 82.4,
        "band": "CRITICAL",
        "layer": 2,
        "role": "DISTRIBUTOR",
        "role_confidence": 0.87,
        "incoming_transactions": 96,
        "outgoing_transactions": 88,
        "first_seen": "2026-09-15T09:14:02",
        "last_seen": "2026-09-29T21:51:30",
        "match_type": "prefix"
      }
    ],
    "total_matches": 7,
    "truncated": false,
    "suggestions": []
  }
}
```


Rules

1. Backed by a **sorted account-number array + binary search** for prefixes and the `account_no → idx` dict for exact hits. It **MUST NOT** issue a DuckDB `LIKE` scan over 2 M rows. Target p95 **< 50 ms**.
2. Empty result ⇒ **200 with** `results: []`, not 404. `suggestions` carries up to 3 Levenshtein ≤ 2 near-matches so the UI can offer "did you mean…".
3. `layer` / `role` here are the **global hints** of 6.3, not case-relative truth. The frontend must label them accordingly.
4. `q` is validated `^[0-9]{2,12}$`. Anything else → `400 INVALID_ACCOUNT`. No user string ever reaches a query string that is concatenated into SQL (section 24).

6.2 `GET /api/accounts/{account_no}/exists`

A 2-field probe (`{"exists": true, "account_idx": 1234}`) so the UI can validate an input box without pulling a full detail payload. Target **< 5 ms**.

6.3 Global hints vs case-relative truth — read this before using `layer` anywhere

```
GLOBAL HINT      computed once at ingest, victim-agnostic
                 answers "does this account LOOK like a collector/distributor/terminal?"
                 fields: role, role_confidence, layer (hint)
                 source: /api/accounts/*, /api/accounts/search

CASE-RELATIVE    computed per trace, victim-anchored
                 answers "in THIS case, what position does it occupy?"
                 fields: layer (1|2|3), hop, role, confidence, reasons
                 source: /api/trace, /api/cases/{case_id}/accounts/{acct}/classification
```

[PS] Layers are victim-relative. The same account can be L1 in one case and L2 in another, and both are true. **[DEC] The UI must never show a global hint inside a case view**, and the API makes that easy by giving them different shapes: a global hint has no `hop` and no `case_id`; a case-relative classification always has both.

---

7. Account Details API

7.1 `GET /api/accounts/{account_no}`

```
{
  "success": true,
  "data": {
    "account": {
      "account_no": "402938475610",
      "account_idx": 1234,
      "primary_ifsc": "HDFC0001234",
      "all_ifsc": ["HDFC0001234"],
      "bank_code": "HDFC",
      "first_seen": "2026-09-15T09:14:02",
      "last_seen": "2026-09-29T21:51:30",
      "active_days": 14
    },

    "statistics": {
      "incoming_transactions": 96,
      "outgoing_transactions": 88,
      "total_transactions": 184,
      "incoming_amount": 250000.00,
      "outgoing_amount": 242000.00,
      "net_amount": 8000.00,
      "unique_senders": 9,
      "unique_receivers": 6,
      "median_incoming_amount": 1850.00,
      "median_outgoing_amount": 2100.00,
      "largest_incoming": 48000.00,
      "largest_outgoing": 51000.00,
      "balance_held_estimate": 8000.00,
      "degree_balance": 0.60,
      "payment_mode_mix": { "UPI": 140, "IMPS": 31, "NEFT": 10, "RTGS": 3 },
      "device_mix": { "Android": 150, "Windows_Browser": 34 },
      "distinct_ip_count": 12,
      "activity_by_hour": { "0": 2, "1": 0, "9": 31 }
    },

    "flow": {
      "pass_through_ratio": 0.93,
      "pass_through_method": "FIFO_TEMPORAL_MATCH",
      "median_hold_seconds": 412,
      "p90_hold_seconds": 1180,
      "matched_in_value": 232500.00,
      "unmatched_in_value": 17500.00,
      "burst_count": 4,
      "max_outgoing_in_window": 11,
      "split_consistency": 0.82
    },

    "risk": {
      "mri": 81.4,
      "band": "CRITICAL",
      "components": {
        "C1_pass_through": { "raw": 0.93, "weighted": 23.25, "weight": 25 },
        "C2_velocity":     { "raw": 0.88, "weighted": 17.60, "weight": 20 },
        "C3_fan_in":       { "raw": 0.71, "weighted": 10.65, "weight": 15 },
        "C4_fan_out":      { "raw": 0.83, "weighted": 12.45, "weight": 15 },
        "C5_terminal":     { "raw": 0.10, "weighted": 1.50,  "weight": 15,
                             "cooccurrence_discount": 1.0 },
        "C6_structure":    { "raw": 0.62, "weighted": 6.20,  "weight": 10 }
      },
      "adjustments": [
        { "rule": "SINGLE_COMPONENT_CAP", "applied": false, "delta": 0 },
        { "rule": "HIGH_VOLUME_SUPPRESSION", "applied": false, "delta": 0 }
      ],
      "reasons": [
        "93% of incoming value was forwarded within a median of 6m 52s (FIFO-matched).",
        "Received from 9 distinct accounts and dispersed to 6, with consistent split sizes.",
        "Outgoing activity concentrated in 4 bursts rather than spread across the window."
      ],
      "calibration_regime": "HEURISTIC_SELF_VALIDATION"
    },

    "classification_hint": {
      "role": "DISTRIBUTOR",
      "confidence": 0.87,
      "layer_hint": 2,
      "is_victim_relative": false,
      "note": "Global hint. Open a case and use the case-relative classification for evidence."
    },

    "terminal_evidence_summary": {
      "terminal_score": 0.10,
      "signal_rows": 0,
      "has_any_signal": false
    },
```

```
    "neighbors": {
      "top_senders":   [ { "account_no": "918273645501", "transactions": 14, "amount": 62000.00 } ],
      "top_receivers": [ { "account_no": "551122334455", "transactions": 11, "amount": 58000.00 } ]
    }
  }
}
```

Rules

1. **Both sides, always.** Incoming and outgoing statistics are both populated for every account, including accounts that happen to have zero on one side (then the counters are `0`, not `null`). This is the API-level expression of "collector and distributor are role descriptors, not exclusive capabilities" (PRD §2.4). **MUST**
2. Served from the in-RAM feature frame + precomputed neighbour top-K. **No DuckDB scan.** Target p95 **< 150 ms**.
3. `components` exposes `raw`, `weight` and `weighted` separately so the UI can draw the contribution bar **and** the explanation without arithmetic of its own.
4. `reasons` are generated server-side from templates bound to the actual numbers. They are the strings that end up in documents; the frontend displays them verbatim and **MUST NOT** compose its own.
5. `balance_held_estimate` is dataset-derived (`in − out`) and is labelled an estimate everywhere it appears, including in the freeze requisition. It is **not** a bank balance.
6. Unknown account → `404 ACCOUNT_NOT_FOUND`, with `suggestions` in `details` when a near match exists.

---

8. Transaction API

8.1 `GET /api/accounts/{account_no}/transactions`

Parameter	Type	Default	Notes
`direction`	enum	`all`	`in` | `out` | `all`
`start_time`	ISO-8601	dataset start	Inclusive
`end_time`	ISO-8601	dataset end	Exclusive
`min_amount`	number	—	Optional
`max_amount`	number	—	Optional
`payment_mode`	string	—	Optional, must be a value from `profile.payment_modes`
`counterparty`	string	—	Optional 12-digit account filter
`signal_only`	bool	`false`	Only rows carrying a terminal signal
`page`	int	1	1-based
`page_size`	int	50	Clamped to 500
`sort`	enum	`time_asc`	`time_asc` | `time_desc` | `amount_desc`

Response


```
{
  "success": true,
  "data": {
    "account_no": "402938475610",
    "direction": "all",
    "transactions": [
      {
        "source_row_id": 123456,
        "transaction_id": "TXN0000123456",
        "transaction_id_is_duplicated": false,
        "sender_account": "918273645501",
        "receiver_account": "402938475610",
        "sender_ifsc": "SBIN0004567",
        "receiver_ifsc": "HDFC0001234",
        "amount": 25000.00,
        "timestamp": "2026-09-22T11:02:41",
        "payment_mode": "UPI",
        "narration": "UPI/P2P/payment",
        "ip_address": "185.220.101.44",
        "device_type": "Android",
        "direction": "in",
        "signals": ["foreign_ip"],
        "row_hash": "9c1f…22ad"
      }
    ],
    "pagination": { "page": 1, "page_size": 50, "total": 184, "total_pages": 4, "has_next": true },
    "totals": { "count": 184, "in_count": 96, "out_count": 88,
                "in_amount": 250000.00, "out_amount": 242000.00 }
  }
}
```


Rules

1. **`source_row_id` is mandatory on every transaction object returned by any endpoint.** It is the immutable CSV row ordinal and the only safe way to point at a row, because `Transaction_ID` repeats. **MUST**
2. `transaction_id_is_duplicated` tells the UI when to render the disambiguation affordance (timestamp + amount + counterparty) next to an ID.

3. `signals` is a list of the terminal signals present **on that row** (`foreign_ip`, `automation_device`, `crypto_narration`, `wallet_narration`). An empty list is the overwhelmingly common case. A non-empty list is **evidence, not a verdict** — the UI must not colour the row "fraud".
4. This endpoint does read DuckDB, but through the `(account_idx, timestamp)` index; it touches the pages for one account, not 2 M rows.
5. `totals` reflects the **filter**, not the page, so the UI can show "184 transactions, ₹2.5 L in / ₹2.42 L out" without a second call.

8.2 `GET /api/transactions/{transaction_id}`

Returns a LIST. Always. Even when there is exactly one match.


```
{
  "success": true,
  "data": {
    "transaction_id": "TXN0000123456",
    "is_duplicated": true,
    "match_count": 2,
    "matches": [
      { "source_row_id": 123456, "timestamp": "2026-09-22T11:02:41", "amount": 25000.00,
        "sender_account": "918273645501", "receiver_account": "402938475610",
        "row_hash": "9c1f…22ad", "duplicate_class": "SAME_ID_DIFFERENT_ATTRIBUTES" },
      { "source_row_id": 871204, "timestamp": "2026-09-24T16:41:08", "amount": 7400.00,
        "sender_account": "551122334455", "receiver_account": "402938475610",
        "row_hash": "41bb…7e90", "duplicate_class": "SAME_ID_DIFFERENT_ATTRIBUTES" }
    ],
    "disambiguation_required": true,
    "citation_hint": "Cite as TXN0000123456 (row 123456, 2026-09-22 11:02:41, ₹25,000.00)."
  }
}
```


`duplicate_class`	Meaning	Handling
`UNIQUE`	ID appears once	Normal
`SAME_ID_DIFFERENT_ATTRIBUTES`	Same ID, different timestamp/amount/parties — **two real events**	Both are evidence; both kept; citations must disambiguate
`EXACT_DUPLICATE`	Every field identical	Both rows preserved; **counted once in aggregates** (`DEDUPE_EXACT_IN_AGGREGATES`), flagged in the profile

Rules

1. **MUST NOT** return a single object, ever. A client written against "one ID → one transaction" is a client that will mis-cite evidence in a freeze requisition. [PS-adjacent, PRD §11.3 D1–D9]
2. **MUST NOT** delete, merge or quarantine any row. Both rows above exist in DuckDB and both are citable.
3. No match → `404 TRANSACTION_NOT_FOUND`.

8.3 `GET /api/rows/{source_row_id}`

The provenance endpoint: give it a row ordinal, get that exact CSV row back, verbatim, plus how it was interpreted.

```
{
  "success": true,
  "data": {
    "source_row_id": 123456,
    "raw": { "Transaction_ID": "TXN0000123456", "Timestamp": "2026-09-22 11:02:41",
             "Sender_Account": "918273645501", "Amount": "25000.00" },
    "normalized": { "amount": 25000.00, "timestamp": "2026-09-22T11:02:41",
                    "sender_idx": 8812, "receiver_idx": 1234 },
    "row_hash": "9c1f…22ad",
    "signals": ["foreign_ip"],
    "used_in": { "edges": 1, "cases": ["CASE-2026-001"] }
  }
}
```


This is what **validator V12** calls: before a document is released, every cited fact is re-read from the source row and compared field by field. It is also the endpoint a sceptical judge is pointed at — "every number in this notice traces back to a line in your CSV, here it is".

---

9. Money Trail / 4-Hop Trace

9.1 `POST /api/trace`

Request

```
{
  "victim_account": "402938475610",
  "case_id": null,
  "max_hops": 4,
  "direction": "down",
  "start_time": null,
  "end_time": null,
  "window_sec": 21600,
  "min_value": null,
  "max_nodes": 800,
  "topk_per_node": 15,
  "initial_amount": null
}
```

Field	Type	Default	Notes
`victim_account`	string	required	12 digits
`case_id`	string|null	`null`	When present, the result is persisted to the case and layers become case-relative
`max_hops`	int	4	1–4; above 4 → `422 TRACE_LIMIT_EXCEEDED`
`direction`	enum	`down`	`down` = where did the money go · `up` = who funded this account (the blind-ID fallback, PRD §17.5)
`start_time` / `end_time`	ISO|null	incident time → +window	Explicit bounds override `window_sec`
`window_sec`	int	`TRACE_WINDOW_SEC` (6 h)	Must be one of `TRACE_WINDOW_PRESETS` or any value ≤ 72 h
`min_value`	number|null	`max(₹100, 0.2% × initial_amount)`	`null` means "use the computed floor" — **not** zero
`max_nodes`	int	800	Clamped to `TRACE_MAX_NODES`
`topk_per_node`	int	15	Clamped
`initial_amount`	number|null	largest inbound credit in window	Seed value for proportional attribution

Response

```
{
  "success": true,
  "data": {
    "trace_id": "trc_01JA8Z9V3K",
    "case_id": "CASE-2026-001",
    "victim": "402938475610",
    "direction": "down",
    "initial_amount": 250000.00,
    "total_traced_value": 250000.00,
    "max_hop": 4,

    "parameters_used": {
      "window_sec": 21600, "min_value": 500.00, "topk_per_node": 15,
      "max_nodes": 800, "hub_degree_cap": 500,
      "start_time": "2026-09-22T11:00:00", "end_time": "2026-09-22T17:00:00",
      "attribution_method": "FIFO"
    },

    "nodes": [
      {
        "account_no": "918273645501",
        "account_idx": 8812,
        "hop": 1,
        "layer": 1,
        "role": "COLLECTOR",
        "role_confidence": 0.91,
        "risk_score": 86.0,
        "band": "CRITICAL",
        "traced_value": 120000.00,
        "traced_value_held": 4200.00,
        "traced_value_forwarded": 115800.00,
        "in_degree_case": 1,
        "out_degree_case": 4,
        "is_terminal": false,
        "is_hub": false,
        "truncated_expansion": false,
        "reasons": ["Received ₹1,20,000 of traced value from the victim within 11 minutes."]
      }
    ],

    "edges": [
      {
        "edge_id": "e_8812_1234",
        "source": "918273645501",
        "target": "402938475610",
        "amount": 50000.00,
        "traced_amount": 50000.00,
        "transaction_count": 2,
        "first_timestamp": "2026-09-22T11:02:41",
        "last_timestamp": "2026-09-22T11:19:03",
        "hop": 1,
        "source_row_ids": [123456, 123981],
        "payment_modes": ["UPI", "IMPS"]
      }
    ],

    "summary": {
      "node_count": 34,
      "edge_count": 51,
      "l1_count": 1, "l2_count": 3, "l3_count": 7,
      "unclassified_count": 23,
      "terminal_value": 94000.00,
      "held_value_total": 38400.00,
      "freezable_accounts": 5,
      "truncated": false,
      "truncation_reason": null,
      "hubs_skipped": 0
    },

    "performance": { "traversal_ms": 38, "enrichment_ms": 54, "total_ms": 97 }
  }
}
```

9.2 Algorithm contract

```
frontier = {victim at hop 0, value = initial_amount}
for hop in 1..max_hops:
    for node in frontier:
        candidate edges = CSR slice for node          ← O(degree), no SQL
        keep edges where  t(edge) > t(parent credit)  ← temporal causality
        attribute value proportionally to edge amount ← FIFO
        drop edges whose attributed value < min_value
        keep top-K edges by attributed value
        if degree(node) > hub_cap and value < material: mark AGGREGATOR, do not expand
        stop if node_count >= max_nodes → truncated = true
```

Rules

1. **p95 ≤ 2 s** [PS]. Measured end-to-end at the HTTP boundary, reported in `performance` on every response and aggregated into `/api/benchmarks`.
2. **MUST use the precomputed CSR.** A normal trace **MUST NOT** scan the 2 M-row fact table. DuckDB is touched only in the enrichment phase, and only for the `source_row_ids` actually on returned edges.
3. **Bounded output, always.** When a limit bites, the response is still 200 with `summary.truncated = true` and a `truncation_reason` of `MAX_NODES` | `MAX_HOPS` | `MIN_VALUE` | `HUB_GUARD`. A partial answer with an honest flag beats an unbounded one. `422 TRACE_LIMIT_EXCEEDED` is reserved for requests that ask for more than the server permits (e.g. `max_hops: 9`).
4. **Window default is 6 hours, not 72** (PRD §7.5): at 72 h the branching factor is ≈16 and hop 4 reaches ≈66,885 accounts — the entire population. A trace that returns everything has told the investigator nothing.
5. `traced_value_held + traced_value_forwarded == traced_value` for every node, within ₹0.01. This identity is a test (E-group), not a hope.
6. Empty trace (victim has no qualifying outbound flow) ⇒ **200 with empty arrays** and `meta.warnings: ["no outbound flow in window; try direction=up or a wider window"]`. The UI turns that into the "view upstream sources" button. Never a 404.
7. Determinism: identical input ⇒ byte-identical output. Tie-breaks sort by `(attributed_value desc, timestamp asc, source_row_id asc)`.

9.3 `GET /api/trace/{trace_id}`

Re-fetch a completed trace without recomputing (LRU of the last 50 traces, **[DEC]**). Used by the timeline, the graph view and document generation so that all three see exactly the same traversal.

---

10. Layer Classification API

10.1 `GET /api/accounts/{account_no}/classification` — global hint


```
{
  "success": true,
  "data": {
    "account_no": "402938475610",
    "is_victim_relative": false,
    "layer_hint": 2,
    "role": "DISTRIBUTOR",
    "confidence": 0.88,
    "evidence": {
      "in_degree": 9,
      "out_degree": 6,
      "degree_balance": 0.60,
      "pass_through_ratio": 0.93,
      "pass_through_method": "FIFO_TEMPORAL_MATCH",
      "median_hold_seconds": 412,
      "burst_count": 4,
      "split_consistency": 0.82,
      "terminal_signal_ratio": 0.02,
      "upstream_risk_mean": 0.74,
      "downstream_risk_mean": 0.41
    },
    "scores": { "collector_score": 0.44, "distributor_score": 0.81, "terminal_score": 0.10 },
    "reasons": [
      "Dispersed 93% of matched incoming value to 6 downstream accounts.",
      "Both receives and sends — role assigned by dominant topology, not by capability.",
      "No terminal signal present on any of its 184 rows."
    ],
    "caveat": "Global hint, victim-agnostic. Use the case-relative endpoint for evidence."
  }
}
```


10.2 `GET /api/cases/{case_id}/accounts/{account_no}/classification` — case-relative

Same shape **plus** the case anchoring, and this is the only form that may appear in a document:


```
{
  "success": true,
  "data": {
    "case_id": "CASE-2026-001",
    "victim_account": "551100220033",
    "account_no": "402938475610",
    "is_victim_relative": true,
    "layer": 2,
    "hop": 2,
    "role": "DISTRIBUTOR",
    "confidence": 0.88,
    "confidence_breakdown": {
      "topology": 0.34, "flow_behaviour": 0.29, "temporal": 0.16,
      "upstream_context": 0.09, "terminal_evidence": 0.00
    },
    "inputs_used": [
      "hop_depth", "in_degree", "out_degree", "degree_balance",
      "pass_through_ratio", "hold_time", "split_consistency",
      "terminal_signal_ratio", "upstream_layer", "downstream_layer"
    ],
    "evidence": { },
    "reasons": [
      "Two hops from the victim account on the traced path.",
      "Received ₹1,20,000 of traced value and forwarded 93% of it within 7 minutes.",
      "Four of its six downstream counterparties carry terminal evidence."
    ],
    "alternative_considered": { "layer": 3, "confidence": 0.31,
                                "rejected_because": "forwarded value exceeds the terminal retention threshold" }
  }
}
```

Rules

1. **All ten inputs are consulted** (PRD §16.1). Classifying from hop depth alone, in-degree alone or out-degree alone is **forbidden** — a unit test (D-group) asserts that a fixture with a decisive single feature and contradicting evidence is not classified by that feature.
2. `confidence_breakdown` sums to `confidence` (±0.01) and must be renderable as a stacked bar.
3. `alternative_considered` is mandatory whenever the runner-up scored ≥ 0.25. This is what turns "the computer said L2" into a defensible opinion.
4. The same account in two cases may legitimately return two different layers. Clients **MUST NOT** cache classification by `account_no` alone; the cache key is `(case_id, account_no)`.
5. An account that is not part of the case's trace → `404` with code `ACCOUNT_NOT_IN_CASE`.

---

11. Graph API

11.1 `GET /api/graph/{account_no}`

Parameter	Type	Default	Notes
`radius`	int	2	1–3 (`GRAPH_MAX_RADIUS`)
`max_nodes`	int	800	Clamped to `GRAPH_MAX_NODES`
`max_edges`	int	1500	Clamped to `GRAPH_MAX_EDGES`
`direction`	enum	`both`	`in` | `out` | `both`
`start_time` / `end_time`	ISO	dataset window	Edge time filter
`min_amount`	number	0	Edge amount filter
`layout`	enum	`dagre`	`dagre` | `cose` | `concentric` — a **hint** returned verbatim; the backend does no layout maths
`case_id`	string	—	When given, node `layer` / `role` are case-relative and traced edges are marked

Response


```
{
  "success": true,
  "data": {
    "center": "402938475610",
    "radius": 2,
    "layout": "dagre",
    "nodes": [
      {
        "id": "402938475610",
        "account_no": "402938475610",
        "layer": 1,
        "role": "COLLECTOR",
        "risk_score": 82.0,
        "band": "CRITICAL",
        "is_center": true,
        "is_terminal": false,
        "is_hub": false,
        "in_degree": 9,
        "out_degree": 6,
        "traced_value": 120000.00,
        "hop": 1,
        "expandable": false,
        "hidden_neighbors": 0
      }
    ],
    "edges": [
      {
        "id": "e_1234_8812",
        "source": "402938475610",
        "target": "918273645501",
        "amount": 50000.00,
        "count": 2,
        "first_timestamp": "2026-09-22T11:02:41",
        "last_timestamp": "2026-09-22T11:19:03",
        "is_traced": true,
        "source_row_ids": [123456, 123981]
      }
    ],
    "stats": { "node_count": 112, "edge_count": 287, "hubs_collapsed": 2,
               "truncated": false, "truncation_reason": null },
    "legend": { "bands": { "LOW": "0-30", "MEDIUM": "30-60", "HIGH": "60-80", "CRITICAL": "80-100" } }
  }
}
```

Rules

1. **The full graph is never returned.** 24,873 nodes / ≈2.0 M edges would destroy the browser and tell the investigator nothing. Every response is a bounded neighbourhood. **MUST**
2. When bounds bite: return 200, set `truncated`, set `hidden_neighbors` and `expandable` per node so the UI can offer a deliberate "expand" click. `422 GRAPH_LIMIT_EXCEEDED` is only for requested limits above the server maxima.
3. Hubs (degree > `hub_degree_cap`, derived from `profile.graph_shape.out_degree.p99 × 3`) are included as nodes but not expanded; they carry `is_hub: true` and are excluded from freeze suggestions.
4. Node and edge shapes are **Cytoscape-ready** but not Cytoscape-specific: flat objects, string ids, no nested `data` wrapper. The adapter lives in the frontend.
5. `is_traced` distinguishes edges on the traced money path from ordinary context edges — the difference between "this is the trail" and "this is the neighbourhood".

11.2 `GET /api/graph/{account_no}/expand`

Expand exactly one collapsed node (`node`, `max_nodes` ≤ 200). Returns only the **new** nodes/edges so the frontend patches its graph instead of re-rendering it.

11.3 `GET /api/graph/rings` (P2)

Ring / cycle detection over the case subgraph:
`{"rings": [{"accounts": [...], "length": 4, "total_amount": 180000.00, "confidence": 0.72, "source_row_ids": [...]}]}`. Bounded to cycles of length ≤ 6 and to the case subgraph — **never** a global cycle enumeration over 2 M edges.

---

12. Timeline API

12.1 `GET /api/timeline`

Parameter	Type	Default	Notes
`account`	string	—	One of `account` / `case_id` / `trace_id` is required
`case_id`	string	—	Timeline of the case's traced edges
`trace_id`	string	—	Timeline of one specific traversal
`start_time` / `end_time`	ISO	dataset window	
`minute_bucket`	int	1	Bucket size in minutes for the histogram (1, 5, 15, 60)
`max_events`	int	5000	Clamped to `TIMELINE_MAX_EVENTS`

Response


```
{
  "success": true,
  "data": {
    "scope": { "case_id": "CASE-2026-001", "account": null },
    "window": { "start": "2026-09-22T11:00:00", "end": "2026-09-22T17:00:00" },
    "minute_bucket": 1,
    "events": [
      {
        "timestamp": "2026-09-22T11:02:41",
        "source": "551100220033",
        "target": "402938475610",
        "amount": 250000.00,
        "hop": 0,
        "source_row_id": 123456,
        "payment_mode": "IMPS",
        "is_traced": true,
        "signals": []
      }
    ],
    "buckets": [
      { "bucket_start": "2026-09-22T11:02:00", "count": 1, "amount": 250000.00, "hop_max": 0 }
    ],
    "cumulative": [
      { "timestamp": "2026-09-22T11:02:41", "value_traced": 250000.00, "accounts_touched": 2 }
    ],
    "stats": { "event_count": 51, "first_event": "2026-09-22T11:02:41",
               "last_event": "2026-09-22T13:44:10", "span_seconds": 9689, "truncated": false }
  }
}
```


Rules

1. **Minute-level filtering is mandatory** — layering happens in minutes, and the PS pattern completes in 3–15 minutes. Bucketing is server-side; the frontend never aggregates 5,000 events itself.
2. `events` is sorted ascending by `(timestamp, source_row_id)` — stable, so playback is reproducible.
3. `cumulative` drives the playback scrubber's "value traced so far" readout without a second pass in the browser.
4. `buckets` is always returned even when `events` is truncated, so the histogram stays accurate while the event list is capped.

13. Risk API

13.1 `GET /api/risk/{account_no}`


```
{
  "success": true,
  "data": {
    "account_no": "402938475610",
    "mri": 82.5,
    "band": "CRITICAL",
    "band_edges": { "LOW": 30, "MEDIUM": 60, "HIGH": 80 },
    "percentile": 99.2,

    "components": {
      "pass_through":       { "raw": 0.94, "weight": 25, "weighted": 23.50 },
      "velocity":           { "raw": 0.88, "weight": 20, "weighted": 17.60 },
      "fan_in":             { "raw": 0.71, "weight": 15, "weighted": 10.65 },
      "fan_out":            { "raw": 0.83, "weight": 15, "weighted": 12.45 },
      "terminal_signals":   { "raw": 0.10, "weight": 15, "weighted": 1.50,
                              "cooccurrence_discount": 1.0, "signals_present": [] },
      "behavioral_anomaly": { "raw": 0.62, "weight": 10, "weighted": 6.20 }
    },

    "adjustments": [
      { "rule": "SINGLE_COMPONENT_CAP", "applied": false, "cap": 40, "delta": 0,
        "explanation": "No single component contributed more than 40 points." },
      { "rule": "HIGH_VOLUME_SUPPRESSION", "applied": false, "delta": 0,
        "explanation": "Account is not in the top 0.1% by volume; 'busy' is not 'guilty'." },
      { "rule": "TERMINAL_SIGNAL_SOLO_CAP", "applied": false, "cap": 45, "delta": 0 }
    ],

    "reasons": [
      "Rapidly dispersed 94% of FIFO-matched incoming value, median hold 6m 52s.",
      "Distributed incoming funds across 6 accounts with consistent split sizes.",
      "Received from 9 distinct accounts inside a 4-hour window."
    ],

    "evidence_rows": [123456, 123981, 124402],
    "calibration_regime": "HEURISTIC_SELF_VALIDATION",
    "computed_at": "2026-10-01T18:05:02",
    "disclaimer": "A risk score indicates investigative priority. It is not a determination of fraud."
  }
}
```


Rules

1. **The backend calculates the MRI. The frontend NEVER calculates the MRI.** The UI renders `mri`, `band`, `components`, `adjustments` and `reasons`. If a designer wants a different weighting, that is a `config.py` change, not a frontend change. **MUST**
2. `sum(weighted) + sum(adjustment deltas) == mri`, within 0.1. Enforced by a test.
3. Every component carries `raw`, `weight` and `weighted` so the explanation is self-contained and auditable in a screenshot.
4. `adjustments` lists rules **even when they did not fire** (`applied: false`). "We checked and it didn't apply" is information; silence is not.
5. `reasons` are human sentences bound to real numbers. **Never** "suspicious pattern detected". Each reason must name a quantity a judge can check against `evidence_rows`.
6. `disclaimer` is returned by the API, not written by the frontend, so it cannot be dropped by a styling decision. [PS]
7. Precomputed at ingest; served from RAM. Target p95 **< 50 ms**.

13.2 `POST /api/risk/batch`

`{"account_nos": ["…", "…"]}` (max 500) → compact `{account_no, mri, band, layer_hint, role}` records, for colouring a whole subgraph in one call instead of N.

13.3 `GET /api/risk/distribution`

Histogram of MRI across all accounts (20 buckets) + band counts + the top-N list. Drives the dashboard's "where does this account sit?" context strip. Precomputed at ingest.

---

14. Terminal Node API

14.1 `GET /api/accounts/{account_no}/terminal-evidence`


```
{
  "success": true,
  "data": {
    "account_no": "774411220099",
    "terminal_score": 0.82,
    "score_basis": "WEIGHTED_EVIDENCE",

    "signals": {
      "foreign_ip":        { "present": true,  "row_count": 3, "weight": 0.25 },
      "automation_device": { "present": true,  "row_count": 3, "weight": 0.25 },
      "crypto_narration":  { "present": true,  "row_count": 3, "weight": 0.20 },
      "wallet_narration":  { "present": false, "row_count": 0, "weight": 0.15 },
      "low_onward_transfer": { "present": true, "value": 0.04, "weight": 0.35 }
    },

    "cooccurrence": {
      "signals_present": 3,
      "independent_signals_estimated": 1,
      "dataset_cooccurrence_rate": 1.0,
      "discount_applied": 0.60,
      "note": "Foreign IP, automation device and crypto narration co-occur on the same rows in this dataset, so the
    },

    "evidence": [
      { "source_row_id": 1884422, "transaction_id": "TXN0001884422",
        "timestamp": "2026-09-22T13:41:55", "amount": 48000.00,
        "signal": "automation_device", "value": "Linux_Script",
        "counterparty": "402938475610" }
    ],

    "interpretation": {
      "verdict": null,
      "statement": "Three co-occurring terminal indicators on 3 of 41 rows, combined with 4% onward transfer, are c
      "requires_corroboration": true
    }
  }
}
```


Rules

1. **A single terminal signal MUST NOT classify an account as malicious.** `Device_Type = "Linux_Script"` on one row produces evidence and a modest score contribution — never a verdict. The `TERMINAL_SIGNAL_SOLO_CAP` (45) exists precisely to make that impossible. [PS, PRD §14.4]
2. `cooccurrence.discount_applied` is driven by the **measured** rate in `profile.json`, not by an assumption baked into code. If the profiler shows the signals are independent, the discount disappears on its own.
3. `interpretation.verdict` is **always** `null` in v1.0. The field exists so no future developer is tempted to put a boolean elsewhere.
4. Terminal signals touch ≈0.13% of rows and plausibly only L3 accounts. An account with **no** signals is not thereby innocent — recall for L1/L2 comes from topology and behaviour. The frontend must not render "no signals" as a green tick.
5. Every item in `evidence` carries a `source_row_id`, so it is citable in a document and re-verifiable through `/api/rows/{id}`.

15. Evidence Basket API

The basket is the investigator's working set: the accounts, rows and edges they have deliberately pinned as relevant. **Documents are generated from the basket, not from whatever happens to be on screen.** [DEC]

15.1 `POST /api/evidence/basket`

Request

```
{
  "case_id": "CASE-2026-001",
  "account_ids": ["402938475610", "918273645501"],
  "source_row_ids": [123456, 123981],
  "transaction_ids": ["TXN0000123456"],
  "edge_ids": ["e_1234_8812"],
  "note": "Primary collector and the two credits that funded it"
}
```

Response — 201

```
{
  "success": true,
  "data": {
    "case_id": "CASE-2026-001",
    "basket_id": "bsk_01JA8ZB4QY",
    "items_added": 12,
    "items_skipped": 1,
    "item_count": 12,
    "skipped": [
      { "ref": "TXN0000123456", "reason": "AMBIGUOUS_TRANSACTION_ID",
        "detail": "ID matches 2 rows; pin a source_row_id instead",
        "candidates": [123456, 871204] }
    ]
  }
}
```

Rules

1. **Every pinned item is normalised to `source_row_id` internally.** A `transaction_id` that resolves to exactly one row is accepted and converted. One that resolves to several is **rejected into `skipped`** with its candidates — the investigator chooses. The basket never guesses which duplicate was meant. **MUST**
2. Adding an item twice is a no-op (`items_skipped`, reason `DUPLICATE_PIN`) — the endpoint is idempotent.
3. Unknown account / row → the item lands in `skipped` with `ACCOUNT_NOT_FOUND` / `TRANSACTION_NOT_FOUND`; the rest of the batch still succeeds. A partially valid batch must not fail wholesale.
4. Cap: `BASKET_MAX_ITEMS` (500) → `422` with code `BASKET_LIMIT_EXCEEDED`.

15.2 `GET /api/evidence/basket/{case_id}`


```
{
  "success": true,
  "data": {
    "case_id": "CASE-2026-001",
    "basket_id": "bsk_01JA8ZB4QY",
    "item_count": 12,
    "items": [
      { "item_id": "itm_0007", "kind": "ROW", "source_row_id": 123456,
        "transaction_id": "TXN0000123456", "timestamp": "2026-09-22T11:02:41",
        "amount": 25000.00, "sender_account": "918273645501",
        "receiver_account": "402938475610", "pinned_at": "2026-10-01T18:12:04",
        "note": null, "verified": true },
      { "item_id": "itm_0008", "kind": "ACCOUNT", "account_no": "918273645501",
        "layer": 1, "role": "COLLECTOR", "mri": 86.0, "pinned_at": "2026-10-01T18:12:04",
        "verified": true }
    ],
    "totals": { "accounts": 4, "rows": 7, "edges": 1, "pinned_value": 182400.00 },
    "verification": { "all_rows_reverified": true, "checked_at": "2026-10-01T18:12:05",
                      "failures": [] }
  }
}
```


`kind` ∈ `ACCOUNT` · `ROW` · `EDGE`. `verified` is the result of re-reading the row from DuckDB and comparing the stored snapshot field by field (**validator V12**). A `false` here blocks document generation.

15.3 `DELETE /api/evidence/basket/{case_id}/{item_id}`

Removes one pinned item → `{"removed": true, "item_count": 11}`. Unknown item → `404 EVIDENCE_ITEM_NOT_FOUND`. **Removing an item from a basket never deletes data** — it un-pins a reference. The source row is immutable.

15.4 `DELETE /api/evidence/basket/{case_id}`

Clears the basket (requires `?confirm=true`, else `400`).

---

16. Case API

A **case** is the unit of investigation: one victim, one incident time, one trace, one basket, zero or more documents. It is also the scope that makes layer classification meaningful.

16.1 `POST /api/cases`

Request

```
{
  "victim_account": "551100220033",
  "incident_time": "2026-09-22T11:00:00",
  "reported_amount": 250000.00,
  "complainant_name": null,
  "fir_number": null,
  "police_station": "Crime Branch, Indore",
  "investigating_officer": null,
  "notes": ""
}
```

Only `victim_account` and `incident_time` are required. The administrative fields are optional at creation and may be filled in later via `PATCH`; they are needed before a freeze requisition can be generated (section 18), and the API says so explicitly rather than silently emitting a blank form.

Response — 201

```
{
  "success": true,
  "data": {
    "case_id": "CASE-2026-001",
    "status": "created",
    "victim_account": "551100220033",
    "incident_time": "2026-09-22T11:00:00",
    "created_at": "2026-10-01T18:10:22",
    "dataset_fingerprint": "sha256:7f3c…b41a",
    "next_step": "POST /api/trace with this case_id"
  }
}
```

Rules

1. `case_id` format: `CASE-{YYYY}-{NNN}`, monotonic per process. **[DEC]**
2. The case records the `dataset_fingerprint` at creation. If the dataset is later re-ingested with a different fingerprint, every endpoint for that case returns `meta.warnings: ["case was created against a different dataset"]` and document generation is blocked. Evidence that refers to rows from another file is not evidence.
3. `incident_time` must fall inside the dataset window → else `400 INVALID_TIME_RANGE` with the valid range in `details`.
4. Case state is in-memory + a JSON sidecar in `DOCUMENT_DIR/cases/` so a backend restart during judging does not lose the demo case. **[DEC]**

16.2 `GET /api/cases/{case_id}`

The one call that rehydrates an entire investigation.

```
{
  "success": true,
  "data": {
    "case": {
      "case_id": "CASE-2026-001", "status": "traced",
      "victim_account": "551100220033", "incident_time": "2026-09-22T11:00:00",
      "reported_amount": 250000.00, "police_station": "Crime Branch, Indore",
      "fir_number": null, "investigating_officer": null,
      "created_at": "2026-10-01T18:10:22", "updated_at": "2026-10-01T18:14:51",
      "dataset_fingerprint": "sha256:7f3c…b41a"
    },

    "victim": { "account_no": "551100220033", "primary_ifsc": "SBIN0004567",
                "outgoing_amount_in_window": 250000.00 },

    "trace": { "trace_id": "trc_01JA8Z9V3K", "node_count": 34, "edge_count": 51,
               "total_traced_value": 250000.00, "max_hop": 4, "truncated": false,
               "parameters_used": { "window_sec": 21600, "min_value": 500.00 } },

    "layers": {
      "l1": [ { "account_no": "918273645501", "role": "COLLECTOR", "mri": 86.0,
                "confidence": 0.91, "traced_value": 120000.00 } ],
      "l2": [ ], "l3": [ ],
      "counts": { "l1": 1, "l2": 3, "l3": 7, "unclassified": 23 }
    },

    "risk_scores": { "918273645501": 86.0, "402938475610": 82.5 },

    "evidence_basket": { "basket_id": "bsk_01JA8ZB4QY", "item_count": 12,
                         "all_verified": true },

    "timeline": { "first_event": "2026-09-22T11:02:41", "last_event": "2026-09-22T13:44:10",
                  "event_count": 51, "span_seconds": 9689 },

    "current_holdings": {
      "method": "TRACED_IN_MINUS_TRACED_OUT",
      "is_estimate": true,
      "total_held": 38400.00,
      "accounts": [
        { "account_no": "774411220099", "held": 18400.00, "layer": 3,
          "freeze_recommended": true, "reason": "Retained 96% of ₹19,200 received; terminal evidence present." }
      ],
      "disclaimer": "Derived from the transaction dataset. Not a bank balance; the bank must confirm."
    },

    "documents": [
      { "document_id": "doc_01JA8ZC9QT", "kind": "CASE_DIARY", "format": "pdf",
        "status": "generated", "generated_at": "2026-10-01T18:16:40",
        "download_url": "/api/documents/doc_01JA8ZC9QT", "validator_passed": true }
    ],

    "readiness": {
      "can_generate_case_diary": true,
      "can_generate_freeze_requisition": false,
      "blockers": ["fir_number is required for a freeze requisition",
                   "investigating_officer is required for a freeze requisition"]
    }
  }
}
```


`status` ∈ `created` → `traced` → `evidence_collected` → `documented`. **`readiness.blockers`** is the contract that stops the UI from offering a button that will fail: the backend states exactly what is missing, in user language.

16.3 Other case routes

Route	Purpose
`GET /api/cases`	List cases (id, victim, status, created_at, document count)
`PATCH /api/cases/{case_id}`	Update administrative fields (`fir_number` , `investigating_officer` , `notes` , `complainant_name`)
`DELETE /api/cases/{case_id}`	Delete the case, its basket and its generated documents. **Never** touches dataset artifacts

Route	Purpose
`GET /api/cases/{case_id}/holdings`	The `current_holdings` block on its own, for the freeze-selection screen

17. Case Diary API

17.1 `POST /api/cases/{case_id}/documents/case-diary`

Request

```
{
  "format": "pdf",
  "sections": ["summary", "trace", "layers", "evidence", "timeline", "methodology"],
  "use_llm": false,
  "language": "en",
  "include_annexures": true
}
```

Field	Type	Default	Notes
`format`	enum	`pdf`	`pdf` | `docx` | `markdown` | `json`
`sections`	string[]	all	Subset/ordering of the standard sections
`use_llm`	bool	`false`	Prose polish only; **ignored** if no LLM is available, with a `meta.warnings` note
`include_annexures`	bool	`true`	Annexure A (transactions), B (accounts), E (methodology & limitations)

Response — 201

```
{
  "success": true,
  "data": {
    "document_id": "doc_01JA8ZC9QT",
    "case_id": "CASE-2026-001",
    "kind": "CASE_DIARY",
    "format": "pdf",
    "status": "generated",
    "download_url": "/api/documents/doc_01JA8ZC9QT",
    "meta_url": "/api/documents/doc_01JA8ZC9QT/meta",
    "size_bytes": 184022,
    "page_count": 7,
    "generated_at": "2026-10-01T18:16:40",
    "generation_ms": 2140,
    "llm_used": false,

    "evidence_object": {
      "fingerprint": "sha256:91ab…77cd",
      "fact_count": 47,
      "source_row_count": 23,
      "accounts": 9
    },

    "validation": {
      "passed": true,
      "validators_run": ["V1","V2","V3","V4","V5","V6","V7","V8","V9","V10","V11","V12"],
      "facts_checked": 47,
      "facts_verified": 47,
      "failures": []
    }
  }
}
```

17.2 Generation contract

```
  1  Load case + basket            → fail CASE_NOT_FOUND / EVIDENCE_BASKET_EMPTY
  2  Build EvidenceObject          → every fact carries source_row_id (section 20)
  3  Run validators V1..V12        → any failure ⇒ 422 EVIDENCE_VALIDATION_FAILED, NO file
  4  Render deterministic template → Jinja2, facts injected as typed values
  5  Optional: LLM prose           → narrative paragraphs only, digits stripped & re-validated
  6  Render PDF/DOCX               → write to DOCUMENT_DIR under a server-generated name
  7  Record document metadata      → return document_id + download_url
```

Rules

1. **Template-first, always.** The document is complete and correct with `use_llm: false`. The LLM is an optional prose polisher, never a data source. [PS, PRD §20]
2. **The LLM MUST NOT produce:** account numbers, transaction IDs, `source_row_id`s, amounts, timestamps, IFSC codes, layer assignments, risk scores, or any evidence fact. Enforced three ways: (a) the prompt receives no raw numbers, only placeholder tokens; (b) a post-generation regex rejects any digit sequence the model emitted; (c) validators re-check every fact against the source rows.
3. **Validation failure produces no file.** The endpoint returns `422 EVIDENCE_VALIDATION_FAILED` listing each failed fact (`{fact_path, expected, found, source_row_id}`). A document that cannot be verified must not exist — half-correct legal paperwork is worse than none.
4. Generation is synchronous with a **10 s** budget (NFR). Beyond that → `504` with `DOCUMENT_GENERATION_FAILED`.
5. Every generated document embeds, in its footer and its metadata: `case_id`, `dataset_fingerprint`, `evidence_object.fingerprint`, generation timestamp, and the tool version. Two runs on the same evidence produce byte-identical documents apart from the timestamp. **[DEC]**
6. Annexure E (methodology & limitations) is **mandatory** and generated, not optional boilerplate: attribution convention (FIFO), trace window, hop limit, the victim-relative nature of layers, and the explicit statement that signals are evidence rather than proof.

---

18. Freeze Requisition API

18.1 `POST /api/cases/{case_id}/documents/freeze-requisition`

Request

```
{
  "format": "pdf",
  "selected_accounts": ["774411220099", "402938475610"],
  "legal_basis": "BNSS_94",
  "addressed_to": { "bank": "HDFC Bank", "branch_ifsc": "HDFC0001234",
                    "nodal_officer": "The Nodal Officer (Cyber)" },
  "requested_action": "FREEZE_AND_REPORT",
  "use_llm": false
}
```

Field	Notes
`selected_accounts`	**Required, non-empty.** The backend never decides on its own which accounts to freeze; it recommends via `current_holdings.freeze_recommended` and the officer selects
`legal_basis`	`BNSS_94` (default) | `BNSS_106` — rendered as "Section 94 BNSS, 2023 (corresponding to Section 91 CrPC, 1973)"
`requested_action`	`FREEZE_AND_REPORT` | `REPORT_ONLY`

Response — 201: same shape as 17.1, plus:

```
{
  "accounts_included": 2,
  "total_amount_requested": 60400.00,
  "per_account": [
    { "account_no": "774411220099", "ifsc": "HDFC0001234",
      "amount_traced": 19200.00, "amount_held_estimate": 18400.00,
      "layer": 3, "mri": 88.0, "source_row_ids": [1884422, 1884510],
      "basis_sentence": "Received ₹19,200 of traced victim funds in two transfers on 22-09-2026 and retained an est
  ],
  "warnings": ["Held amounts are dataset-derived estimates and are described as such in the document."]
}
```

Rules

1. **Every account in the requisition must be in the case trace and in the evidence basket.** An account that is in neither → `422 EVIDENCE_VALIDATION_FAILED` naming it. You cannot freeze an account you have not evidenced.
2. **Hub/aggregator accounts are rejected by default** (`is_hub: true`) with a clear error; overriding requires `allow_hub_accounts: true` and the document then carries a visible caution. Freezing a payment aggregator because it has high in-degree is the single most damaging false positive this system could cause.
3. Amounts are stated as **traced** and **estimated held**, never as "balance". The sentence template says so in words.
4. Administrative blockers (`fir_number`, `investigating_officer`, `police_station`) are enforced here, not silently left blank → `422` with `details.missing_fields`.
5. The legal citation string is produced by the backend from `legal_basis`, so it cannot drift between screens and documents. The BNSS↔CrPC mapping is recorded in PRD §21 and flagged there as requiring mentor/legal confirmation. **[ASSUME]**

19. Document Download

19.1 `GET /api/documents/{document_id}`

Returns the generated file as a byte stream.

Format	Content-Type
`pdf`	`application/pdf`
`docx`	`application/vnd.openxmlformats-officedocument.wordprocessingml.document`
`markdown`	`text/markdown; charset=utf-8`
`json`	`application/json` (the EvidenceObject itself)

```
Content-Disposition: attachment; filename="CASE-2026-001_case_diary_20261001.pdf"
X-Document-Id: doc_01JA8ZC9QT
X-Evidence-Fingerprint: sha256:91ab…77cd
X-Request-ID: req_01JA8Z7K2M9QX4
```

Rules

1. `document_id` **is an opaque server-generated ULID. It is never a path.** The server maps id → stored file through its own registry; the client cannot name a file on disk. Any attempt to pass a path separator is rejected before the lookup. **MUST** (section 24)
2. The resolved path is `realpath`-checked to be inside `DOCUMENT_DIR`; a check failure is logged as a security event and returns a plain `404`.
3. The download filename is composed server-side from `case_id`, kind and date, then sanitised to `[A-Za-z0-9._-]`.
4. Unknown id → `404 DOCUMENT_NOT_FOUND`. Expired/cleaned file → `410` with `DOCUMENT_EXPIRED`.
5. `GET /api/documents/{document_id}/meta` returns the JSON metadata (kind, format, size, page count, validation summary, evidence fingerprint) so the UI can show a card without downloading the bytes.
6. `GET /api/cases/{case_id}/documents` lists all documents for a case.

---

20. Evidence Object

The `EvidenceObject` is the **single source of every factual token** in every generated document. If a value is not in this object, it cannot appear in a document. If it is in this object, it carries a `source_row_id`.

20.1 Pydantic model

```python
from pydantic import BaseModel, Field, field_validator
from datetime import datetime
from typing import Literal

class SourceRef(BaseModel):
    """Provenance anchor. Transaction_ID is NOT unique; source_row_id is."""
    source_row_id: int
    transaction_id: str
    transaction_id_is_duplicated: bool = False
    row_hash: str

class TransactionFact(BaseModel):
    ref: SourceRef
    timestamp: datetime
    sender_account: str
    receiver_account: str
    sender_ifsc: str
    receiver_ifsc: str
    amount: float
    payment_mode: str
    narration: str
    ip_address: str
    device_type: str
    signals: list[str] = []

class AccountFact(BaseModel):
    account_no: str
    account_idx: int
    primary_ifsc: str
    bank_code: str
    first_seen: datetime
    last_seen: datetime
    incoming_transactions: int
    outgoing_transactions: int
    incoming_amount: float
    outgoing_amount: float
    unique_senders: int
    unique_receivers: int
    evidence_row_ids: list[int]

class EdgeFact(BaseModel):
    source_account: str
    target_account: str
    amount: float
    traced_amount: float
    transaction_count: int
    first_timestamp: datetime
    last_timestamp: datetime
    source_row_ids: list[int]

class LayerFact(BaseModel):
    account_no: str
    layer: Literal[1, 2, 3]
    hop: int
    role: Literal["COLLECTOR", "DISTRIBUTOR", "TERMINAL", "MIXED"]
    confidence: float = Field(ge=0, le=1)
    is_victim_relative: Literal[True] = True      # documents use case-relative only
    reasons: list[str]
    inputs_used: list[str]

class RiskFact(BaseModel):
    account_no: str
    mri: float = Field(ge=0, le=100)
    band: Literal["LOW", "MEDIUM", "HIGH", "CRITICAL"]
    components: dict[str, float]
    adjustments: list[dict]
    reasons: list[str]
    calibration_regime: str

class TerminalEvidenceFact(BaseModel):
    account_no: str
    terminal_score: float
    signals_present: list[str]
    cooccurrence_discount: float
    evidence_rows: list[SourceRef]
    verdict: None = None                          # structurally impossible to assert fraud

class TotalsFact(BaseModel):
    total_traced_value: float
    terminal_value: float
    held_value_estimate: float
    accounts_in_trace: int
    rows_cited: int
    is_estimate: bool = True
```

```python
class EvidenceObject(BaseModel):
    # identity
    case_id: str
    evidence_fingerprint: str                    # sha256 over the canonical JSON
    dataset_fingerprint: str                     # ties evidence to one exact CSV
    schema_version: Literal["1.0"] = "1.0"

    # subject
    victim_account: str
    incident_time: datetime
    trace_id: str

    # facts — every one traceable
    accounts: list[AccountFact]
    transactions: list[TransactionFact]
    edges: list[EdgeFact]
    layers: list[LayerFact]
    risk_scores: list[RiskFact]
    terminal_evidence: list[TerminalEvidenceFact]
    timestamps: dict[str, datetime]              # first_event, last_event, trace_run_at
    totals: TotalsFact
    source_row_ids: list[int]                    # union of every row cited anywhere

    # provenance of the analysis itself
    trace_parameters: dict
    config_snapshot: dict
    generation_timestamp: datetime
    generator_version: str

    @field_validator("source_row_ids")
    @classmethod
    def rows_must_be_unique_and_sorted(cls, v: list[int]) -> list[int]:
        assert len(v) == len(set(v)), "duplicate source_row_id in evidence"
        return sorted(v)
```

20.2 Validators V1–V12

#	Validator	Fails when
V1	Account existence	A cited account is absent from the account dictionary
V2	Transaction existence	A cited row is absent from DuckDB
V3	Amount fidelity	A rendered amount differs from the source row
V4	Timestamp fidelity	A rendered timestamp differs from the source row
V5	IFSC fidelity	A rendered IFSC differs from the source row
V6	Arithmetic consistency	Totals do not equal the sum of their parts
V7	Layer consistency	A layer assertion contradicts the trace's own output
V8	No-digits-from-LLM	The LLM segment contains any digit sequence
V9	Template completeness	A placeholder survived into the output
V10	Fingerprint match	`dataset_fingerprint` differs from the currently loaded dataset
V11	**Citation resolvability**	A cited `Transaction_ID` resolves to more than one row without date-time + amount + counterparty disambiguation
V12	**Row re-verification**	Re-reading each cited `source_row_id` from DuckDB does not match the evidence snapshot field by field

V8 in practice: the prose model never sees a number. It is handed `{{AMOUNT_1}}`-style tokens, and the renderer substitutes the real values after the digit check. A model that cannot see a number cannot hallucinate one.

21. Evaluation API

> **Standing fact: we hold no ground-truth labels.** The problem statement describes 1,500 mule + 23,500 regular accounts; we observe ≈24,873 accounts, which is consistent with that population and proves nothing. Mentor question Q1 is OPEN. Every endpoint below is written so that the honest answer is the easy one.

21.1 `GET /api/eval/summary`

When labels are unavailable (the current state):

```
{
  "success": true,
  "data": {
    "status": "GROUND_TRUTH_UNAVAILABLE",
    "reason": "No labelled mule/non-mule set has been provided for this dataset (mentor Q1, OPEN).",
    "accounts_evaluated": 24873,
    "mules_detected": 1418,
    "precision": null,
    "recall": null,
    "f1": null,
    "false_positives": null,
    "false_negatives": null,
    "available_alternatives": [
      "/api/eval/self-validation  — synthetic-injection metrics, clearly labelled",
      "/api/risk/distribution     — score distribution and band counts"
    ]
  }
}
```

When labels are available (a mentor supplies them, or a synthetic run is active): the same shape with `status: "LABELS_PROVIDED"` or `"SYNTHETIC_INJECTION"`, populated numbers, and a mandatory `regime` field.

`regime` value	Meaning
`GROUND_TRUTH`	Official labels from the organisers
`SYNTHETIC_INJECTION`	Our own injected mule patterns — measures sensitivity, not accuracy
`HEURISTIC_SELF_VALIDATION`	Rule-vs-rule agreement; the weakest claim

Rules

1. **MUST NOT fabricate metrics.** `null` is a correct value. A number with no labels behind it is a lie that a judge will find in thirty seconds. [PS-adjacent, PRD §24.2]
2. `regime` is **required** on every populated metric response. A number without its regime is not publishable — not in the API, not in the UI, not on a slide.
3. `mules_detected` (a count of accounts above the MRI threshold) is always available and always safe: it is a statement about our own output, not about truth.

21.2 `GET /api/eval/confusion-matrix`

```
{ "status": "GROUND_TRUTH_UNAVAILABLE", "tp": null, "tn": null, "fp": null, "fn": null,
  "threshold": 60, "regime": null }
```

21.3 `GET /api/eval/layers`

Per-layer precision/recall/F1 and a 3×3 layer confusion matrix — same `null`-when-unlabelled discipline. **Layer metrics must be reported separately from account-level metrics**, because terminal signals can only find L3: an aggregate score hides the fact that L1/L2 recall comes entirely from topology and behaviour.

21.4 `GET /api/eval/threshold-sweep`

MRI threshold τ from 0 to 100 in steps of 5 with the resulting detected-count curve, plus precision/recall per τ **if and only if** labels exist. Without labels it returns the detected-count curve alone — which is still genuinely useful for picking an operating point, and is honestly labelled as such.

21.5 `POST /api/eval/self-validation` (P2)

Runs the synthetic mule injector over a copy of the graph and reports detection sensitivity. The response hard-codes `regime: "SYNTHETIC_INJECTION"` and a `caveat` string that the UI must display next to any number taken from it.

---

22. Benchmark API

22.1 `GET /api/benchmarks`


```
{
  "success": true,
  "data": {
    "measured": false,
    "hardware": null,
    "dataset_fingerprint": "sha256:7f3c…b41a",

    "ingestion_ms": null,
    "peak_memory_mb": null,
    "feature_generation_ms": null,
    "graph_build_ms": null,
    "trace_p95_ms": null,
    "trace_p50_ms": null,
    "search_p95_ms": null,
    "account_details_p95_ms": null,
    "graph_p95_ms": null,
    "document_generation_ms": null,

    "targets": {
      "ingestion_ms": 60000, "peak_memory_mb": 6144, "trace_p95_ms": 2000,
      "search_p95_ms": 1000, "document_generation_ms": 10000
    },
    "sample_counts": { "trace": 0, "search": 0, "graph": 0 },
    "measured_at": null,
    "note": "null means not yet measured. Estimates are never published here."
  }
}
```


Rules

1. `null` **until measured.** No placeholder, no "approximately", no number carried over from a different machine. The PRD's performance tables and this endpoint share the same discipline. **MUST**
2. Latency figures are populated from a rolling reservoir of real request timings (the `processing_ms` that middleware already records), not from a synthetic microbenchmark. `sample_counts` says how much evidence is behind each percentile; a p95 over 3 samples is reported with its sample count so nobody over-reads it.
3. Ingestion and build timings come from `manifest.stage_timings_ms` — measured during the real run.
4. `targets` are the PS/NFR numbers, returned alongside so the UI can render target-vs-actual without hard-coding anything.
5. `POST /api/benchmarks/run` (P2) executes the standard suite (N traces over sampled victims, N searches, N graph pulls), writes `docs/benchmarks.json`, and flips `measured` to `true`.

---

23. API Error Codes

23.1 The closed set

The frontend branches on `error.code` and nothing else. Adding a code is a contract change.

Code	HTTP	When	`details` carries	Frontend behaviour
`DATASET_NOT_LOADED`	503	No artifacts loaded yet	`{ "ingest_required": true }`	Route to the ingest/admin screen
`INGESTION_IN_PROGRESS`	409	A second ingest attempted	`{ "active_job_id": "…", "progress": 72 }`	Show the existing job's progress
`JOB_NOT_FOUND`	404	Unknown `job_id`	`{ "job_id": "…" }`	Clear the stale poll loop
`ACCOUNT_NOT_FOUND`	404	Account absent from the dataset	`{ "account_no": "…", "suggestions": ["…"] }`	"Did you mean…"
`ACCOUNT_NOT_IN_CASE`	404	Account is not in this case's trace	`{ "case_id": "…", "account_no": "…" }`	Offer the global view instead
`CASE_NOT_FOUND`	404	Unknown `case_id`	`{ "case_id": "…" }`	Back to case list
`TRANSACTION_NOT_FOUND`	404	No row for that `Transaction_ID` / `source_row_id`	`{ "transaction_id": "…" }`	Inline error
`AMBIGUOUS_TRANSACTION_ID`	409	An operation needs one row but the ID matches several	`{ "transaction_id": "…", "candidates": [123456, 871204] }`	**Prompt the user to choose a row**
`INVALID_ACCOUNT`	400	Not 12 digits / wrong charset	`{ "field": "account_no", "pattern": "^[0-9]{12}$" }`	Field-level validation message
`INVALID_TIME_RANGE`	400	`start >= end`, unparseable, or outside the dataset window	`{ "dataset_window": { "start": "…", "end": "…" } }`	Reset the picker to the valid range
`INVALID_PARAMETER`	400	Any other bad query/body value	`{ "field": "…", "allowed": [...] }`	Field-level message
`TRACE_LIMIT_EXCEEDED`	422	Request asks beyond server maxima	`{ "field": "max_hops", "requested": 9, "max": 4 }`	Clamp and retry, or explain
`GRAPH_LIMIT_EXCEEDED`	422	Same, for graph bounds	`{ "field": "max_nodes", "requested": 5000, "max": 800 }`	Clamp and retry
`BASKET_LIMIT_EXCEEDED`	422	Basket over `BASKET_MAX_ITEMS`	`{ "item_count": 500, "max": 500 }`	Prompt to prune
`EVIDENCE_BASKET_EMPTY`	422	Document requested with nothing pinned	`{ "case_id": "…" }`	Guide to pin evidence first

Code	HTTP	When	`details` carries	Frontend behaviour
`EVIDENCE_VALIDATION_FAILED`	422	A validator V1–V12 failed	`{ "failures": [ { "validator": "V12", "fact_path": "transactions[3].amount", "expected": 25000.0, "found": 24000.0, "source_row_id": 123456 } ] }`	**Show the failing facts. Do not retry blindly**
`DOCUMENT_GENERATION_FAILED`	500/504	Renderer or template error, or timeout	`{ "stage": "pdf_render", "reason": "…" }`	Offer `format: markdown` fallback
`DOCUMENT_NOT_FOUND`	404	Unknown `document_id`	`{ "document_id": "…" }`	Refresh the document list
`DOCUMENT_EXPIRED`	410	File cleaned from disk	`{ "document_id": "…" }`	Offer regeneration
`EVIDENCE_ITEM_NOT_FOUND`	404	Unknown basket `item_id`	`{ "item_id": "…" }`	Refresh the basket
`GROUND_TRUTH_UNAVAILABLE`	200	Evaluation requested with no labels	`{ "reason": "…" }`	**Render the honest banner, not an empty chart**
`PROFILE_MISMATCH`	200	`profile-check` disagrees with PRD §7.2	`{ "mismatches": [ { "key": "…", "expected": …, "actual": … } ] }`	Persistent admin warning
`FILE_TOO_LARGE`	413	Upload over `UPLOAD_MAX_BYTES`	`{ "size_bytes": …, "max_bytes": … }`	Field error
`UNSUPPORTED_FILE_TYPE`	400	Not `.csv` / `.csv.gz`	`{ "filename": "…", "allowed": [".csv", ".csv.gz"] }`	Field error
`LLM_UNAVAILABLE`	200	`use_llm: true` but no local model	`{ "fallback": "template_only" }`	Info toast; **generation still succeeds**
`INTERNAL_ERROR`	500	Anything unexpected	`{ "request_id": "…" }`	"Something went wrong" + the request id

Note the two deliberate 200s. `GROUND_TRUTH_UNAVAILABLE` and `PROFILE_MISMATCH` are states of the world, not request failures. Returning 4xx for them would push the frontend into an error path and make the honest answer look like a bug.

23.2 Exception hierarchy

```
class AbhedyaError(Exception):
    code: str = "INTERNAL_ERROR"
    http_status: int = 500
    message: str = "An unexpected error occurred."
    def __init__(self, message=None, **details):
        self.message = message or self.message
        self.details = details

class NotReadyError(AbhedyaError):      code, http_status = "DATASET_NOT_LOADED", 503
class AccountNotFound(AbhedyaError):    code, http_status = "ACCOUNT_NOT_FOUND", 404
class AmbiguousTransaction(AbhedyaError): code, http_status = "AMBIGUOUS_TRANSACTION_ID", 409
class LimitExceeded(AbhedyaError):      code, http_status = "TRACE_LIMIT_EXCEEDED", 422
class EvidenceInvalid(AbhedyaError):    code, http_status = "EVIDENCE_VALIDATION_FAILED", 422
```

One handler converts any `AbhedyaError` into the envelope; one catch-all converts everything else into `INTERNAL_ERROR` **after logging the traceback with the request id**. Routers raise typed errors; they never build error JSON.

---

24. Security Requirements

This platform handles financial-crime data and produces documents that can freeze a citizen's money. The security posture is "local-first, least surface, backend-authoritative".

24.1 Mandatory controls

#	Control	Implementation
1	**Validate all inputs**	Pydantic models on every request; `account_no` `^[0-9]{12}$`; enums for every categorical; bounded ints with `ge` / `le`; ISO-8601 datetimes
2	**Never execute user-supplied SQL**	No endpoint accepts SQL, a column name, an ORDER BY fragment or a filter expression. All DuckDB access is parameterised (`?` binding) inside the data layer. Sort/filter options are **enums mapped to fixed clauses**
3	**Never expose filesystem paths**	No absolute or relative path appears in any response, error, log line shown to the user, or document metadata. Files are addressed by opaque ULIDs
4	**Never expose credentials or config internals**	`/api/health` and `/api/ingest/manifest` publish a curated `config_snapshot` (analysis thresholds only) — never paths, never environment
5	**Restrict document paths**	`document_id → path` via an in-process registry; the resolved path is `realpath`-verified to sit under `DOCUMENT_DIR`; traversal attempts return 404 and log a security event
6	**Sanitise filenames**	Upload names are discarded; storage names are server-generated. Download names are built from `case_id` +kind+date and filtered to `[A-Za-z0-9._-]`, max 120 chars
7	**Limit graph size**	`GRAPH_MAX_NODES` 800, `GRAPH_MAX_EDGES` 1500, `GRAPH_MAX_RADIUS` 3, hub guard — enforced server-side regardless of what the client asks
8	**Limit pagination**	`PAGE_SIZE_MAX` 500, `SEARCH_LIMIT_MAX` 100, `TIMELINE_MAX_EVENTS` 5000 — clamped with a warning, never unbounded
9	**Validate account IDs**	Checked against the account dictionary before any query; an unknown id never reaches the data layer
10	**Validate uploaded files**	Extension allow-list, size cap, streamed write, header/alias check, `sha256` recorded. The CSV is parsed by DuckDB's reader — never `eval`, never `pickle`
11	**Never trust frontend risk scores**	Scores are read from the feature store by account id. The API has **no** endpoint that accepts an MRI value
12	**Never trust frontend layer classifications**	Same: layers come from the trace. No endpoint accepts a layer assignment
13	**Backend is authoritative**	Any client-supplied analytical value is ignored, not merged

24.2 Additional hardening [DEC]

- **Request size cap** on JSON bodies (1 MB) — array fields (`account_ids`, `source_row_ids`) are additionally length-capped.
- **Rate limiting** on the expensive routes (`/api/trace`, `/api/graph/*`, document generation): a simple in-process token bucket, 10 req/s per route. It protects the demo from a double-click storm more than from an attacker.
- **No PII beyond the dataset.** The backend stores no complainant personal data except the optional strings the officer types into the case; those appear only in that case's documents.

- **Masked display support** (PRD SR-08): `GET /api/accounts/{no}?mask=true` returns `XXXX XXXX 5610` in `display_account_no` while leaving `account_no` intact for API use — for screenshots and slides.
- **Audit log**: every document generation writes a line with `request_id`, `case_id`, `document_id`, `evidence_fingerprint`, accounts included and validator result. This is the artefact that answers "who generated what, from which data".
- **No outbound network on the critical path.** The only optional egress is `localhost:11434` (Ollama). With Wi-Fi off, every endpoint in this document still works.

24.3 Explicitly out of scope for v1.0

Authentication, authorisation, multi-tenancy, TLS, and encryption at rest are **not implemented**, because the deployment model is a single investigator's laptop with the dataset already on disk. **This is a stated limitation, not an oversight** — §29 lists it, and the production path (local auth + per-case ACL + encrypted artifact directory) is named there. Saying this plainly is better than a security theatre endpoint that a judge can poke a hole in.

---

25. Backend Directory Structure

```
backend/
└── abhedya/
    ├── main.py                 # app factory, lifespan, middleware, router mount
    ├── config.py               # ALL thresholds and limits, env-overridable
    ├── state.py                # AppState singleton: graph, features, dicts, duckdb conn
    ├── errors.py               # AbhedyaError hierarchy + handlers + envelope
    ├── schemas.py              # shared Pydantic models (envelope, pagination, enums)
    │
    ├── api/
    │   ├── health.py           # GET /health
    │   ├── ingest.py           # POST /ingest · GET /ingest/status · /profile · /manifest
    │   ├── accounts.py         # search · details · exists · classification · terminal-evidence
    │   ├── transactions.py     # account transactions · /transactions/{id} · /rows/{row_id}
    │   ├── trace.py            # POST /trace · GET /trace/{trace_id}
    │   ├── graph.py            # /graph/{account} · /expand · /rings
    │   ├── timeline.py         # GET /timeline
    │   ├── risk.py             # /risk/{account} · /risk/batch · /risk/distribution
    │   ├── evidence.py         # basket CRUD
    │   ├── cases.py            # case CRUD · holdings · case-relative classification
    │   ├── documents.py        # case-diary · freeze-requisition · download · meta
    │   └── evaluation.py       # /eval/* · /benchmarks
    │
    ├── ingest/
    │   ├── loader.py           # streamed CSV → DuckDB, source_row_id assignment
    │   ├── normalize.py        # types, alias map, trimming, row_hash, signal mask
    │   ├── profile.py          # FR-A05 profiler + profile-check (writes profile.json)
    │   ├── duplicates.py       # D1-D9 duplicate policy, repeat histogram, classes
    │   ├── encode.py           # account dictionary, int32 indices
    │   ├── manifest.py         # fingerprint, row conservation (NFR-11), timings
    │   └── jobs.py             # background job runner, stage/progress reporting
    │
    ├── graph/
    │   ├── csr.py              # build/load CSR forward+reverse, degree arrays, hubs
    │   ├── traverse.py         # bounded BFS, value attribution, temporal ordering
    │   ├── subgraph.py         # k-radius extraction, bounding, collapse metadata
    │   └── rings.py            # bounded ring/cycle detection (P2)
    │
    ├── detect/
    │   ├── features.py         # ~45 dual-sided per-account features
    │   ├── passthrough.py      # FIFO temporal matching (NOT total-in vs total-out)
    │   ├── rules.py            # rule cards R1-R10, suppressors
    │   ├── scoring.py          # MRI components, caps, bands, reason strings
    │   ├── layers.py           # 10-input classifier, confidence, alternatives
    │   └── terminal.py         # terminal evidence, co-occurrence discount
    │
    ├── evidence/
    │   ├── builder.py          # trace + basket → EvidenceObject
    │   ├── schema.py           # the Pydantic models of section 20
    │   ├── validator.py        # V1-V12
    │   └── basket.py           # basket persistence and normalisation to source_row_id
    │
    ├── docs/
    │   ├── diary.py            # case diary assembly
    │   ├── requisition.py      # freeze requisition assembly
    │   ├── pdf.py              # HTML→PDF rendering
    │   ├── llm.py              # optional Ollama prose, digit-stripped
    │   └── templates/          # case_diary.html.j2 · freeze_requisition.html.j2 · annexures
    │
    └── tests/
        ├── test_ingest.py      # A-group: row conservation, duplicates, profile-check
        ├── test_graph.py       # B-group: CSR correctness, bounds, hub guard
        ├── test_detect.py      # C-group: features, pass-through, MRI caps, suppressors
        ├── test_layers.py      # D-group: 10 inputs, forbidden single-feature rules
        ├── test_trace.py       # E-group: attribution identity, window, determinism
        ├── test_documents.py   # F-group: validators, lying-LLM stub, re-verification
        ├── test_api.py         # envelope, error codes, status mapping, limits
        └── fixtures/           # 1,000-row deterministic CSV + golden JSON responses
```

Module rules [DEC]

1. `api/*` modules contain **no analysis logic** — they validate, call a service, and return a model.
2. Only `ingest/*`, `graph/*`, `detect/*` and `evidence/*` may touch DuckDB or NumPy arrays.
3. `state.py` is the only global; everything else receives what it needs as an argument.
4. Nothing imports from `docs/*` except `api/documents.py`.

5. Every public function has type hints; `schemas.py` models are shared between routers and tests so a contract drift breaks a test.

---

26. Frontend–Backend Contract

26.1 Complete endpoint table

Method	Endpoint	Purpose	Input	Output	P
GET	`/api/health`	Liveness + load state	—	status, counts, flags	P0
POST	`/api/ingest`	Start dataset build	`file` or `path`, `force_rebuild`	`job_id`, `poll_url`	P0
GET	`/api/ingest/status/{job_id}`	Poll job	path `job_id`	status, stage, progress	P0
GET	`/api/ingest/profile`	FR-A05 profile	—	full `profile.json`	P0
GET	`/api/ingest/manifest`	Build fingerprint	—	fingerprint, counts, timings	P0
GET	`/api/accounts/search`	Find accounts	`q`, `limit`, `min_risk`, `layer`, `sort`	results[], suggestions	P0
GET	`/api/accounts/{account_no}/exists`	Cheap existence probe	path	`exists`, `account_idx`	P0
GET	`/api/accounts/{account_no}`	Full account view	path, `mask`	account, statistics, flow, risk, hint, neighbors	P0
GET	`/api/accounts/{account_no}/transactions`	Paged rows	direction, time, amount, mode, page	transactions[], pagination, totals	P0
GET	`/api/accounts/{account_no}/classification`	Global role hint	path	layer_hint, role, confidence, evidence	P0
GET	`/api/accounts/{account_no}/terminal-evidence`	Terminal signals	path	score, signals, co-occurrence, evidence[]	P0
GET	`/api/transactions/{transaction_id}`	Resolve an ID	path	**matches[] (always a list)**	P0
GET	`/api/rows/{source_row_id}`	Provenance of one row	path	raw, normalized, used_in	P0
POST	`/api/trace`	4-hop money trail	victim, hops, window, limits	nodes[], edges[], summary, performance	P0
GET	`/api/trace/{trace_id}`	Re-fetch a trace	path	same as POST	P0
GET	`/api/graph/{account_no}`	Bounded subgraph	radius, limits, time, `case_id`	nodes[], edges[], stats	P0
GET	`/api/graph/{account_no}/expand`	Expand one node	`node`, `max_nodes`	new nodes[]/edges[]	P1
GET	`/api/graph/rings`	Ring/cycle detection	`case_id`, max length	rings[]	P2
GET	`/api/timeline`	Event stream + buckets	account/case/trace, window, bucket	events[], buckets[], cumulative[]	P1

Method	Endpoint	Purpose	Input	Output	P
GET	`/api/risk/{account_no}`	MRI + explanation	path	mri, band, components, adjustments, reasons	P0
POST	`/api/risk/batch`	Scores for many	`account_nos[]` (≤500)	compact records[]	P0
GET	`/api/risk/distribution`	Score histogram	—	buckets, band counts, top-N	P1
POST	`/api/evidence/basket`	Pin evidence	case_id, ids	items_added, skipped[]	P1
GET	`/api/evidence/basket/{case_id}`	Read basket	path	items[], totals, verification	P1
DELETE	`/api/evidence/basket/{case_id}/{item_id}`	Un-pin one item	path	removed, item_count	P1
DELETE	`/api/evidence/basket/{case_id}`	Clear basket	`confirm=true`	cleared	P1
POST	`/api/cases`	Create a case	victim, incident_time, admin fields	case_id, next_step	P1
GET	`/api/cases`	List cases	`page`	cases[]	P1
GET	`/api/cases/{case_id}`	Rehydrate a case	path	case, trace, layers, basket, holdings, documents, readiness	P1
PATCH	`/api/cases/{case_id}`	Update admin fields	FIR, officer, notes	updated case	P1
DELETE	`/api/cases/{case_id}`	Delete a case	path	deleted	P1
GET	`/api/cases/{case_id}/holdings`	Freeze candidates	path	accounts[], disclaimer	P1
GET	`/api/cases/{case_id}/accounts/{account_no}/classification`	**Case-relative layer**	path	layer, hop, role, confidence breakdown, reasons	P0
GET	`/api/cases/{case_id}/documents`	Documents of a case	path	documents[]	P1
POST	`/api/cases/{case_id}/documents/case-diary`	Generate diary	format, sections, use_llm	document_id, validation	P1
POST	`/api/cases/{case_id}/documents/freeze-requisition`	Generate requisition	format, selected_accounts, legal_basis	document_id, per_account[]	P1
GET	`/api/documents/{document_id}`	Download file	path	**binary**	P1
GET	`/api/documents/{document_id}/meta`	File metadata	path	kind, size, pages, validation	P1
GET	`/api/eval/summary`	Detection metrics	—	status/regime + metrics or `null`	P2
GET	`/api/eval/confusion-matrix`	TP/TN/FP/FN	`threshold`	matrix or `null`	P2
GET	`/api/eval/layers`	Per-layer metrics	—	per-layer metrics or `null`	P2
GET	`/api/eval/threshold-sweep`	τ curve	step	curve[]	P2

Method	Endpoint	Purpose	Input	Output	P
POST	`/api/eval/self-validation`	Synthetic-injection run	params	metrics + `SYNTHETIC_INJECTION` regime	P2
GET	`/api/benchmarks`	Measured performance	—	measured values or `null` + targets	P2
POST	`/api/benchmarks/run`	Run the suite	iterations	populated benchmarks	P2

45 routes (method + path pairs). Nothing outside this table exists in v1.0.

26.2 Worked examples for the twelve critical endpoints

For each: **A** request · **B** success · **C** error. All values are illustrative.

1 · `GET /api/health`

A

```
curl -s http://localhost:8000/api/health
```

B — 200

```
{ "success": true,
  "data": { "status": "healthy", "dataset_loaded": true, "artifacts_loaded": true,
            "graph_loaded": true, "features_loaded": true, "accounts": 24873,
            "transactions": 2000000, "profile_verified": true,
            "ground_truth_available": false, "llm_available": false },
  "meta": { "request_id": "req_01JA8Z7K2M9QX4", "processing_ms": 2, "api_version": "1.0" } }
```

C — 200 with a state, never an error

```
{ "success": true,
  "data": { "status": "unavailable", "dataset_loaded": false, "accounts": 0,
            "transactions": 0, "ingest_job": { "active": false, "last_job_id": null } },
  "meta": { "request_id": "req_01JA8Z7K2M9QY1", "processing_ms": 1, "api_version": "1.0" } }
```

2 · `POST /api/ingest`

A

```
curl -s -X POST http://localhost:8000/api/ingest \
     -F "path=VoidHacks8_MuleAccount_2M_Transactions.csv" \
     -F "force_rebuild=false"
```

B — 202

```
{ "success": true,
  "data": { "job_id": "ingest_20261001_001", "status": "started",
            "filename": "VoidHacks8_MuleAccount_2M_Transactions.csv",
            "size_bytes": 274019328, "sha256": "7f3c…b41a",
            "poll_url": "/api/ingest/status/ingest_20261001_001" },
  "meta": { "request_id": "req_01JA8ZAA10", "processing_ms": 88, "api_version": "1.0" } }
```

C — 409

```
{ "success": false,
  "error": { "code": "INGESTION_IN_PROGRESS",
             "message": "An ingestion job is already running.",
             "details": { "active_job_id": "ingest_20261001_001", "progress": 72 },
             "hint": "Poll /api/ingest/status/ingest_20261001_001." },
  "meta": { "request_id": "req_01JA8ZAB22", "processing_ms": 3, "api_version": "1.0" } }
```

3 · `GET /api/ingest/status/{job_id}`

A

```
curl -s http://localhost:8000/api/ingest/status/ingest_20261001_001
```

B — 200

```
{ "success": true,
  "data": { "job_id": "ingest_20261001_001", "status": "running",
            "stage": "building_graph", "stage_index": 6, "stage_count": 9,
            "progress": 72, "rows_processed": 1440000, "total_rows": 2000000,
            "elapsed_ms": 38200,
            "warnings": ["2252 duplicate Transaction_ID values detected; all rows preserved"],
            "error": null },
  "meta": { "request_id": "req_01JA8ZAC33", "processing_ms": 1, "api_version": "1.0" } }
```

C — 200 with a failed job (the job failed; the request did not)

```
{ "success": true,
  "data": { "job_id": "ingest_20261001_002", "status": "failed", "stage": "reading",
            "progress": 4,
            "error": { "code": "UNSUPPORTED_FILE_TYPE",
                       "message": "Expected 11 columns; found 9.",
                       "details": { "missing": ["IP_Address", "Device_Type"] } } },
  "meta": { "request_id": "req_01JA8ZAD44", "processing_ms": 1, "api_version": "1.0" } }
```

4 · `GET /api/accounts/search`

A

```
curl -s "http://localhost:8000/api/accounts/search?q=4029&limit=5&sort=risk_desc"
```

B — 200

```
{ "success": true,
  "data": { "query": "4029",
            "results": [ { "account_no": "402938475610", "account_idx": 1234,
                           "primary_ifsc": "HDFC0001234", "risk_score": 82.4,
                           "band": "CRITICAL", "layer": 2, "role": "DISTRIBUTOR",
                           "role_confidence": 0.87, "incoming_transactions": 96,
                           "outgoing_transactions": 88, "match_type": "prefix" } ],
            "total_matches": 7, "truncated": false, "suggestions": [] },
  "meta": { "request_id": "req_01JA8ZAE55", "processing_ms": 11, "api_version": "1.0" } }
```

C — 400

```
{ "success": false,
  "error": { "code": "INVALID_ACCOUNT",
             "message": "Search query must be 2 to 12 digits.",
             "details": { "field": "q", "value": "40a9", "pattern": "^[0-9]{2,12}$" } },
  "meta": { "request_id": "req_01JA8ZAF66", "processing_ms": 2, "api_version": "1.0" } }
```

5 · `GET /api/accounts/{account_no}`

A

```
curl -s http://localhost:8000/api/accounts/402938475610
```

B — 200 (abridged; full shape in section 7.1)


```
{ "success": true,
  "data": { "account": { "account_no": "402938475610", "account_idx": 1234,
                         "primary_ifsc": "HDFC0001234", "first_seen": "2026-09-15T09:14:02" },
            "statistics": { "incoming_transactions": 96, "outgoing_transactions": 88,
                            "incoming_amount": 250000.0, "outgoing_amount": 242000.0,
                            "unique_senders": 9, "unique_receivers": 6 },
            "flow": { "pass_through_ratio": 0.93, "pass_through_method": "FIFO_TEMPORAL_MATCH",
                      "median_hold_seconds": 412 },
            "risk": { "mri": 81.4, "band": "CRITICAL" },
            "classification_hint": { "role": "DISTRIBUTOR", "confidence": 0.87,
                                     "layer_hint": 2, "is_victim_relative": false } },
  "meta": { "request_id": "req_01JA8ZAG77", "processing_ms": 38, "api_version": "1.0" } }
```


C — 404


```
{ "success": false,
  "error": { "code": "ACCOUNT_NOT_FOUND",
             "message": "Account was not found in the loaded dataset.",
             "details": { "account_no": "402938475619", "suggestions": ["402938475610"] },
             "hint": "Use /api/accounts/search for near matches." },
  "meta": { "request_id": "req_01JA8ZAH88", "processing_ms": 4, "api_version": "1.0" } }
```


6 · `POST /api/trace`

A

```
curl -s -X POST http://localhost:8000/api/trace \
  -H 'Content-Type: application/json' \
  -d '{"victim_account":"551100220033","case_id":"CASE-2026-001","max_hops":4,"window_sec":21600}'
```

B — 200 (abridged; full shape in section 9.1)


```
{ "success": true,
  "data": { "trace_id": "trc_01JA8Z9V3K", "case_id": "CASE-2026-001",
            "victim": "551100220033", "initial_amount": 250000.0,
            "total_traced_value": 250000.0, "max_hop": 4,
            "parameters_used": { "window_sec": 21600, "min_value": 500.0,
                                 "topk_per_node": 15, "attribution_method": "FIFO" },
            "nodes": [ { "account_no": "918273645501", "hop": 1, "layer": 1,
                         "role": "COLLECTOR", "role_confidence": 0.91, "risk_score": 86.0,
                         "traced_value": 120000.0, "traced_value_held": 4200.0,
                         "traced_value_forwarded": 115800.0 } ],
            "edges": [ { "edge_id": "e_8812_1234", "source": "551100220033",
                         "target": "918273645501", "amount": 120000.0,
                         "traced_amount": 120000.0, "transaction_count": 2, "hop": 1,
                         "source_row_ids": [123456, 123981] } ],
            "summary": { "node_count": 34, "edge_count": 51, "l1_count": 1, "l2_count": 3,
                         "l3_count": 7, "terminal_value": 94000.0, "truncated": false },
            "performance": { "traversal_ms": 38, "enrichment_ms": 54, "total_ms": 97 } },
  "meta": { "request_id": "req_01JA8ZAJ99", "processing_ms": 97, "api_version": "1.0" } }
```


C — 422


```
{ "success": false,
  "error": { "code": "TRACE_LIMIT_EXCEEDED",
             "message": "max_hops must be between 1 and 4.",
             "details": { "field": "max_hops", "requested": 9, "max": 4 },
             "hint": "Hop 4 already reaches the practical limit of this dataset." },
  "meta": { "request_id": "req_01JA8ZAK10", "processing_ms": 2, "api_version": "1.0" } }
```

> **Also note the empty-but-valid case:** a victim with no qualifying outbound flow returns **200** with `nodes: []`, `edges: []` and `meta.warnings: ["no outbound flow in window; try direction=up or a wider window"]`. The UI turns this into the upstream-trace fallback (PRD §17.5), which protects the highest-weighted judging criterion.

7 · `GET /api/graph/{account_no}`

A

```
curl -s "http://localhost:8000/api/graph/402938475610?radius=2&max_nodes=300&case_id=CASE-2026-001"
```

B — 200

```
{ "success": true,
  "data": { "center": "402938475610", "radius": 2, "layout": "dagre",
            "nodes": [ { "id": "402938475610", "account_no": "402938475610", "layer": 2,
                         "role": "DISTRIBUTOR", "risk_score": 82.0, "band": "CRITICAL",
                         "is_center": true, "is_hub": false, "in_degree": 9,
                         "out_degree": 6, "expandable": false, "hidden_neighbors": 0 } ],
            "edges": [ { "id": "e_1234_8812", "source": "402938475610",
                         "target": "918273645501", "amount": 50000.0, "count": 2,
                         "is_traced": true, "source_row_ids": [123456, 123981] } ],
            "stats": { "node_count": 112, "edge_count": 287, "hubs_collapsed": 2,
                       "truncated": false } },
  "meta": { "request_id": "req_01JA8ZAL11", "processing_ms": 64, "api_version": "1.0" } }
```

C — 422

```
{ "success": false,
  "error": { "code": "GRAPH_LIMIT_EXCEEDED",
             "message": "max_nodes must not exceed 800.",
             "details": { "field": "max_nodes", "requested": 5000, "max": 800 },
             "hint": "Use radius=1 and expand nodes on demand." },
  "meta": { "request_id": "req_01JA8ZAM12", "processing_ms": 2, "api_version": "1.0" } }
```

8 · `GET /api/risk/{account_no}`

A

```
curl -s http://localhost:8000/api/risk/402938475610
```

B — 200 (abridged; full shape in section 13.1)

```
{ "success": true,
  "data": { "account_no": "402938475610", "mri": 82.5, "band": "CRITICAL",
            "band_edges": { "LOW": 30, "MEDIUM": 60, "HIGH": 80 }, "percentile": 99.2,
            "components": { "pass_through": { "raw": 0.94, "weight": 25, "weighted": 23.5 },
                            "terminal_signals": { "raw": 0.10, "weight": 15, "weighted": 1.5,
                                                  "cooccurrence_discount": 1.0,
                                                  "signals_present": [] } },
            "adjustments": [ { "rule": "TERMINAL_SIGNAL_SOLO_CAP", "applied": false,
                               "cap": 45, "delta": 0 } ],
            "reasons": ["Rapidly dispersed 94% of FIFO-matched incoming value, median hold 6m 52s."],
            "calibration_regime": "HEURISTIC_SELF_VALIDATION",
            "disclaimer": "A risk score indicates investigative priority. It is not a determination of fraud." },
  "meta": { "request_id": "req_01JA8ZAN13", "processing_ms": 9, "api_version": "1.0" } }
```

C — 503

```
{ "success": false,
  "error": { "code": "DATASET_NOT_LOADED",
             "message": "No dataset has been ingested yet.",
             "details": { "ingest_required": true },
             "hint": "POST /api/ingest, then poll /api/ingest/status/{job_id}." },
  "meta": { "request_id": "req_01JA8ZAP14", "processing_ms": 1, "api_version": "1.0" } }
```

9 · `POST /api/cases`

A

```
curl -s -X POST http://localhost:8000/api/cases -H 'Content-Type: application/json' \
  -d '{"victim_account":"551100220033","incident_time":"2026-09-22T11:00:00","reported_amount":250000}'
```

B — 201

```
{ "success": true,
  "data": { "case_id": "CASE-2026-001", "status": "created",
            "victim_account": "551100220033", "incident_time": "2026-09-22T11:00:00",
            "created_at": "2026-10-01T18:10:22", "dataset_fingerprint": "sha256:7f3c…b41a",
            "next_step": "POST /api/trace with this case_id" },
  "meta": { "request_id": "req_01JA8ZAQ15", "processing_ms": 6, "api_version": "1.0" } }
```

C — 400

```
{ "success": false,
  "error": { "code": "INVALID_TIME_RANGE",
             "message": "incident_time falls outside the loaded dataset window.",
             "details": { "incident_time": "2025-03-04T11:00:00",
                          "dataset_window": { "start": "2026-09-15T00:00:11",
                                              "end": "2026-09-29T23:59:52" } } },
  "meta": { "request_id": "req_01JA8ZAR16", "processing_ms": 3, "api_version": "1.0" } }
```

10 · `POST /api/evidence/basket`

A

```
curl -s -X POST http://localhost:8000/api/evidence/basket -H 'Content-Type: application/json' \
  -d '{"case_id":"CASE-2026-001","account_ids":["918273645501"],"source_row_ids":[123456,123981]}'
```

B — 201

```
{ "success": true,
  "data": { "case_id": "CASE-2026-001", "basket_id": "bsk_01JA8ZB4QY",
            "items_added": 3, "items_skipped": 0, "item_count": 12, "skipped": [] },
  "meta": { "request_id": "req_01JA8ZAS17", "processing_ms": 14, "api_version": "1.0" } }
```

C — 201 with a partial skip (**the important case**)

```
{ "success": true,
  "data": { "case_id": "CASE-2026-001", "basket_id": "bsk_01JA8ZB4QY",
            "items_added": 2, "items_skipped": 1, "item_count": 14,
            "skipped": [ { "ref": "TXN0000123456", "reason": "AMBIGUOUS_TRANSACTION_ID",
                           "detail": "ID matches 2 rows; pin a source_row_id instead",
                           "candidates": [123456, 871204] } ] },
  "meta": { "request_id": "req_01JA8ZAT18", "processing_ms": 16, "api_version": "1.0" } }
```

11 · `POST /api/cases/{case_id}/documents/case-diary`

A

```
curl -s -X POST http://localhost:8000/api/cases/CASE-2026-001/documents/case-diary \
  -H 'Content-Type: application/json' -d '{"format":"pdf","use_llm":false}'
```

B — 201

```
{ "success": true,
  "data": { "document_id": "doc_01JA8ZC9QT", "case_id": "CASE-2026-001",
            "kind": "CASE_DIARY", "format": "pdf", "status": "generated",
            "download_url": "/api/documents/doc_01JA8ZC9QT", "size_bytes": 184022,
            "page_count": 7, "generation_ms": 2140, "llm_used": false,
            "evidence_object": { "fingerprint": "sha256:91ab…77cd", "fact_count": 47,
                                 "source_row_count": 23 },
            "validation": { "passed": true, "facts_checked": 47, "facts_verified": 47,
                            "failures": [] } },
  "meta": { "request_id": "req_01JA8ZAU19", "processing_ms": 2140, "api_version": "1.0" } }
```

C — 422 (**no file is produced**)

```
{ "success": false,
  "error": { "code": "EVIDENCE_VALIDATION_FAILED",
             "message": "3 of 47 facts failed verification. No document was generated.",
             "details": { "failures": [
               { "validator": "V12", "fact_path": "transactions[3].amount",
                 "expected": 25000.0, "found": 24000.0, "source_row_id": 123456 },
               { "validator": "V11", "fact_path": "transactions[7].transaction_id",
                 "reason": "resolves to 2 rows without disambiguation",
                 "candidates": [871204, 871998] } ] },
             "hint": "Re-pin the ambiguous citation by source_row_id and regenerate." },
  "meta": { "request_id": "req_01JA8ZAV20", "processing_ms": 310, "api_version": "1.0" } }
```

12 · `POST /api/cases/{case_id}/documents/freeze-requisition`

A

```
curl -s -X POST http://localhost:8000/api/cases/CASE-2026-001/documents/freeze-requisition \
  -H 'Content-Type: application/json' \
  -d '{"format":"pdf","selected_accounts":["774411220099"],"legal_basis":"BNSS_94"}'
```

B — 201

```
{ "success": true,
  "data": { "document_id": "doc_01JA8ZD1RM", "kind": "FREEZE_REQUISITION", "format": "pdf",
            "status": "generated", "download_url": "/api/documents/doc_01JA8ZD1RM",
            "accounts_included": 1, "total_amount_requested": 18400.0,
            "per_account": [ { "account_no": "774411220099", "ifsc": "HDFC0001234",
                               "amount_traced": 19200.0, "amount_held_estimate": 18400.0,
                               "layer": 3, "mri": 88.0,
                               "source_row_ids": [1884422, 1884510] } ],
            "validation": { "passed": true, "facts_checked": 22, "facts_verified": 22 },
            "warnings": ["Held amounts are dataset-derived estimates and are described as such in the document."] },
  "meta": { "request_id": "req_01JA8ZAW21", "processing_ms": 1980, "api_version": "1.0" } }
```

C — 422

```
{ "success": false,
  "error": { "code": "EVIDENCE_VALIDATION_FAILED",
             "message": "Cannot generate a freeze requisition: required case fields are missing and one selected ac
             "details": { "missing_fields": ["fir_number", "investigating_officer"],
                          "rejected_accounts": [ { "account_no": "999900001111",
                                                   "reason": "IS_HUB",
                                                   "out_degree": 402,
                                                   "note": "Likely a payment aggregator. Override with allow_hub_ac
             "hint": "PATCH /api/cases/CASE-2026-001 with the FIR number and officer, then retry." },
  "meta": { "request_id": "req_01JA8ZAX22", "processing_ms": 12, "api_version": "1.0" } }
```

27. Frontend Data Flow

27.1 The investigation path

```
User enters victim account
         │
         ▼
POST /api/cases                          → case_id          (201)
         │
         ▼
POST /api/trace  {case_id}               → trace_id, nodes, edges, layers, summary
         │
         ▼
GET  /api/graph/{account}?case_id=…      → bounded subgraph for rendering
         │
         ▼
GET  /api/risk/{account}                 → MRI, components, reasons
         │
         ▼
GET  /api/cases/{case_id}/accounts/{account}/classification
GET  /api/accounts/{account}/terminal-evidence
         │
         ▼
Frontend renders graph  (Cytoscape) + timeline + risk panel
         │
         ▼
User clicks a node
         │
         ▼
GET  /api/accounts/{account}             → full inspector payload
GET  /api/accounts/{account}/transactions?page=1
         │
         ▼
User adds evidence
         │
         ▼
POST /api/evidence/basket  {case_id, account_ids, source_row_ids}
         │
         ▼
Generate document
         │
         ▼
POST /api/cases/{case_id}/documents/case-diary
POST /api/cases/{case_id}/documents/freeze-requisition
         │
         ▼
GET  /api/documents/{document_id}        → PDF bytes
```

27.2 Bootstrap path (every page load)

```
GET /api/health
      │
      ├── status = "healthy"      → enter the investigation UI
      ├── status = "loading"      → poll /api/ingest/status every 2 s, show progress
      ├── status = "degraded"     → enter the UI + show the banner
      │                             (profile_verified=false or llm_available=false)
      │
      └── status = "unavailable"  → ingest screen only
```

27.3 Call-sequencing rules for the frontend

#	Rule
1	**Never call** `/api/trace` **before** `/api/cases` in an investigation flow. A trace without a case produces layers that cannot be cited in a document
2	**Always pass** `case_id` to `/api/graph` and the classification endpoint once a case exists, so the UI shows case-relative layers rather than global hints
3	Use `POST /api/risk/batch` to colour a subgraph — **not** N calls to `/api/risk/{account}`

#	Rule
4	Poll `/api/ingest/status` at 2 s during ingest, `/api/health` at 60 s otherwise. Nothing else polls
5	Cache `/api/ingest/profile` for the session, keyed by `dataset_fingerprint`; invalidate when `meta.dataset_fingerprint` changes
6	Cache classification by `(case_id, account_no)`, never by `account_no` alone
7	Treat `meta.warnings` as renderable — truncation and estimate caveats arrive there
8	On any error, read `error.code`; render `error.message`; log `meta.request_id`. Never parse `message` for logic
9	Before offering a document button, read `case.readiness.blockers` and disable with the blocker text as the tooltip
10	Re-fetch the case (`GET /api/cases/{case_id}`) after any mutation instead of patching local state from a POST response

27.4 What the frontend stores

Allowed	Forbidden
`case_id`, `trace_id`, `basket_id`, `document_id`	Any derived risk score, layer, or role it computed itself
The last response payloads, as a render cache	DuckDB/Parquet/CSR handles — there are none to hold
UI state: selected node, zoom, playback position, filters	Thresholds (they live in `config.py` and arrive in responses)
`dataset_fingerprint`, for cache invalidation	Account→layer maps that outlive a case

28. Non-Negotiable Backend Rules

#	Rule	Enforced by
1	**The frontend never accesses the database directly.** No SQL endpoint, no file route into `data/`, no debug passthrough	Architecture review + section 24 control 2
2	**The frontend never calculates the MRI**	No endpoint accepts a score; `test_api.py` asserts the response carries `mri`, `components` and `reasons`
3	**The frontend never determines L1/L2/L3**	No endpoint accepts a layer; layers originate in `detect/layers.py` only
4	**The frontend never performs 4-hop traversal**	The graph API returns bounded neighbourhoods; adjacency is never exported wholesale
5	**The backend is the source of truth**	Any analytical field in a request body is ignored, not merged
6	**All evidence must be traceable to `source_row_id`**	`EvidenceObject` types require `SourceRef`; validators V11/V12
7	**Duplicate `Transaction_ID` values must not cause evidence loss**	D1–D9; `/api/transactions/{id}` returns a list; `AMBIGUOUS_TRANSACTION_ID`; no row is ever dropped
8	**Never scan 2 M rows for a normal account or trace request**	Query plans reviewed; `test_api.py` asserts p95 budgets on the fixture dataset
9	**Use precomputed indexes / features / CSR on the hot path**	Ingest stages 4–7 exist precisely so request time does no aggregation
10	**Graph responses must be bounded**	`GRAPH_MAX_*`, `TRACE_MAX_NODES`, hub guard — clamped server-side regardless of the request
11	**The LLM is never the source of factual evidence**	Digit-free prompting + V8 digit check + V1–V12 re-verification
12	**No fabricated evaluation metrics**	`null` + `GROUND_TRUTH_UNAVAILABLE` + mandatory `regime` on every populated metric
13	**Every expensive operation exposes timing**	`meta.processing_ms` from middleware; `performance{}` on trace; `stage_timings_ms` on ingest
14	**Every API has typed Pydantic request/response models**	`response_model=` on every route; `test_api.py` validates golden payloads against the models
15	**All thresholds live in `config.py`**	A magic number outside `config.py` is a defect ticket

Three additional rules this contract adds [DEC]:

#	Rule	Why
16	**Every account response carries both incoming and outgoing figures**	"Collector" and "distributor" are roles, not capabilities (PRD §2.4). An API that returns only one side encodes the error in the data model
17	**No endpoint returns a verdict field**	`terminal_evidence.interpretation.verdict` is permanently `null`. A suspicious signal is evidence, never proof
18	**A truncated result is always flagged, never silently trimmed**	`truncated`, `truncation_reason`, `hidden_neighbors`, `meta.warnings`. Silent truncation in an investigation tool is a correctness bug

29. Implementation Notes

29.1 Application state and startup

```
@dataclass
class AppState:
    # dataset
    loaded: bool = False
    fingerprint: str | None = None
    window: tuple[datetime, datetime] | None = None

    # storage handles (never leave the process)
    duck: duckdb.DuckDBPyConnection | None = None

    # graph (NumPy, int32)
    out_indptr: np.ndarray | None = None
    out_indices: np.ndarray | None = None
    out_amount: np.ndarray | None = None
    out_first_ts: np.ndarray | None = None
    in_indptr: np.ndarray | None = None
    in_indices: np.ndarray | None = None

    # lookups
    acct_to_idx: dict[str, int] = field(default_factory=dict)
    idx_to_acct: np.ndarray | None = None
    acct_sorted: np.ndarray | None = None          # prefix search

    # detection
    features: pl.DataFrame | None = None           # ~24.9k x ~45
    mri: np.ndarray | None = None
    band: np.ndarray | None = None
    role_hint: np.ndarray | None = None

    # profiling / provenance
    profile: dict | None = None
    manifest: dict | None = None
    hub_degree_cap: int = 500

    # session objects
    cases: dict[str, Case] = field(default_factory=dict)
    traces: LRU = field(default_factory=lambda: LRU(50))
    documents: dict[str, DocumentRecord] = field(default_factory=dict)
    jobs: dict[str, IngestJob] = field(default_factory=dict)
```

Startup (`lifespan`): if `ARTIFACT_DIR` holds a complete artifact set whose manifest fingerprint matches the raw file, load it (target **< 5 s**) and serve immediately. Otherwise start `unavailable` and wait for `POST /api/ingest`. **The backend never auto-ingests 2 M rows at import time** — a server that takes 60 s to answer `/api/health` looks broken during a demo.

Atomic swap: a rebuild writes to `ARTIFACT_DIR/.staging/`, then rebinds the `AppState` fields under a lock. Requests in flight keep the old arrays; the next request sees the new ones. No partially built state is ever visible.

29.2 Performance budget per endpoint

Endpoint	Target p95	Why it is achievable
`/api/health`	5 ms	Counter reads
`/api/accounts/search`	50 ms	Binary search over a sorted array
`/api/accounts/{no}`	150 ms	One feature-frame row + precomputed top-K neighbours
`/api/risk/{no}`	50 ms	Precomputed vector lookup
`/api/accounts/{no}/transactions`	300 ms	Indexed DuckDB read of one account's pages

Endpoint	Target p95	Why it is achievable
`/api/trace`	**2,000 ms [PS]**	CSR traversal (tens of ms) + enrichment; the budget is spent on evidence retrieval and serialisation
`/api/graph/{no}`	500 ms	Bounded BFS + slice
`/api/timeline`	400 ms	Pre-sorted event array of the trace
document generation	10,000 ms	Template render + PDF

These are **budgets, not measurements**. `/api/benchmarks` returns `null` for each until `make bench` measures it.

29.3 Testing matrix (what `tests/` must cover)

Group	Focus	Representative cases
A Ingest	Row conservation, duplicates, profiling	`rows_out == rows_in`; 2,252-surplus fixture preserves all rows; `profile-check` fails loudly on a mutated fixture; a 9-column CSV fails the job, not the server
B Graph	CSR correctness, bounds	CSR matches a NetworkX reference on a 1,000-node fixture; `max_nodes` always respected; hub guard triggers at the p99-derived cap
C Detection	Features, pass-through, MRI	FIFO matching beats ratio-of-totals on a crafted case; `sum(weighted)+adjustments == mri`; `TERMINAL_SIGNAL_SOLO_CAP` prevents a lone `Linux_Script` row from reaching CRITICAL; high-volume suppression fires on a merchant-like fixture
D Layers	Ten inputs, forbidden rules	An account with hop depth 1 but distributor topology is **not** forced to L1; `confidence_breakdown` sums to `confidence`; the same account yields different layers in two cases
E Trace	Attribution, window, determinism	`held + forwarded == traced` per node; 72 h window on the real distribution hits `truncated`; identical inputs give byte-identical output; empty trace returns 200
F Documents	Validators, anti-hallucination	A **lying LLM stub** that injects a fake account number is caught by V8; a mutated DuckDB row is caught by V12; an ambiguous `Transaction_ID` citation is caught by V11; a validation failure leaves **no file on disk**
API	Contract	Every route returns the envelope; every error code round-trips; limits clamp with a warning; `response_model` validation passes for golden payloads

Fixture discipline: all tests run against a deterministic 1,000-row CSV committed to `tests/fixtures/`, including a hand-built duplicate-ID pair, a signal-bearing row, a hub account and a four-hop chain. No test depends on the 2 M file.

29.4 What each developer takes from this document

Role	Read first	You now know
Backend developer	§2, §3, §23, §25, §29.1	Routes, envelope, error hierarchy, module layout, state object, startup and swap semantics
Frontend developer	§3, §26, §27	Every payload shape, every error code and what to do with it, the exact call order, what you may and may not cache
Detection developer	§7, §9, §10, §13, §14	What the API must expose: dual-sided features, FIFO pass-through, ten classifier inputs, MRI components with caps, terminal evidence with a co-occurrence discount
Document developer	§17, §18, §20	The full `EvidenceObject`, validators V1–V12, the digit-free LLM contract, and that a validation failure must produce no file

Role	Read first	You now know
Test developer	§23, §28, §29.3	Error-code coverage, the fifteen non-negotiable rules as assertions, the six test groups and the fixture discipline

29.5 Known limitations of v1.0 (stated, not hidden)

1. **No authentication or authorisation.** Single-investigator local deployment (§24.3).
2. **Case and job state are in-process** with a JSON sidecar for cases; a restart loses active ingest jobs.
3. **Single dataset at a time.** A second ingest replaces the first; cases from the previous fingerprint are blocked from document generation rather than silently re-interpreted.
4. **Evaluation is unlabelled.** Everything in §21 returns `GROUND_TRUTH_UNAVAILABLE` until that changes.
5. **Benchmarks are unmeasured.** Everything in §22 is `null` until `make bench` runs on the real file.
6. **Held amounts are dataset-derived estimates**, never bank balances, everywhere they appear.
7. **The BNSS/CrPC citation mapping is asserted and awaiting legal confirmation** (PRD §21, mentor question).

BACKEND MVP IMPLEMENTATION ORDER

Build in this order. Each tier is demo-complete on its own, so a stall at any point still leaves something that works.

P0 — the system is useless without these

Order	Item	Endpoints	Done when
1	**Health**	`/api/health`	Returns `unavailable` cleanly with no dataset loaded
2	**Ingestion**	`POST /api/ingest`, `/api/ingest/status/{job_id}`	2 M rows → artifacts, async, with honest stage progress and `rows_out == rows_in`
3	**Profile**	`/api/ingest/profile`, `/api/ingest/manifest`	`profile.json` produced and `profile-check` run; every PRD §7 figure verified or corrected
4	**Account search**	`/api/accounts/search`, `/exists`	Prefix search p95 < 50 ms, with suggestions
5	**Account details**	`/api/accounts/{no}`, `/transactions`, `/api/transactions/{id}`, `/api/rows/{id}`	Both sides populated; transaction ID returns a **list**; every row carries `source_row_id`
6	**Trace**	`POST /api/trace`, `GET /api/trace/{id}`	4 hops, 6 h window, value attribution, bounded, p95 ≤ 2 s, honest truncation flags
7	**Graph**	`/api/graph/{no}`	Bounded subgraph renders in Cytoscape; hubs collapsed, not expanded
8	**Risk**	`/api/risk/{no}`, `/api/risk/batch`, `/distribution`	MRI with components, caps, reasons and the disclaimer
9	**Classification**	`/api/accounts/{no}/classification`, `/api/cases/{id}/accounts/{no}/classification`, `/terminal-evidence`	Ten inputs, confidence breakdown, alternative considered, case-relative form available

P0 exit criterion: blind account number in → bounded graph, layers, risk and reasons out, in under two seconds, entirely offline. That is the highest-weighted judging criterion, and it does not need a single P1 endpoint.

P1 — the system becomes an investigation tool

Order	Item	Endpoints
10	**Timeline**	`/api/timeline`
11	**Evidence basket**	`POST/GET/DELETE /api/evidence/basket/*`
12	**Case management**	`/api/cases` CRUD, `/holdings`, `/documents`
13	**Case diary**	`POST /api/cases/{id}/documents/case-diary`, `/api/documents/{id}`, `/meta`
14	**Freeze requisition**	`POST /api/cases/{id}/documents/freeze-requisition`

P1 exit criterion: victim ID to a validated, downloadable freeze requisition in under two minutes, with every fact re-verified against its source row.

P2 — the system becomes credible under questioning

Order	Item	Endpoints
15	**Evaluation**	`/api/eval/summary`, `/confusion-matrix`, `/layers`, `/threshold-sweep`
16	**Benchmark dashboard**	`GET /api/benchmarks`, `POST /api/benchmarks/run`
17	**Rings & cycles**	`/api/graph/rings`, `/api/graph/{no}/expand`
18	**Optional local LLM prose**	`use_llm: true` path in both document endpoints

P2 exit criterion: every number on screen can state where it came from — measured, self-validated, or unavailable — and the system says which.

> **One sentence for the backend team:** the frontend is allowed to be beautiful and wrong; the backend is not allowed to be either. Every number it emits must be traceable to a row, every bound must be enforced server-side, and every gap — no labels, no benchmarks, an ambiguous transaction ID — must be returned as an explicit state rather than papered over with a plausible value.

Document control

Version	Date	Author	Change
1.0	2026-10-02	Technical Lead (M1)	Initial backend architecture and API contract. 45 routes, standard envelope, closed error-code set, `EvidenceObject` with validators V1–V12, security controls, module layout, test matrix and MVP implementation order. Aligned to PRD v2.1 and the observed dataset profile.

अभेद्य चक्र — the wheel that cannot be broken. The contract is the wheel: the frontend renders, the backend proves.